ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

BÁO CÁO ĐỒ ÁN
MÔN THIẾT KẾ HỆ THỐNG SOC
Đề tài: THIẾT KẾ VÀ XÂY DỰNG BỘ GIA TỐC PHẦN CỨNG SHA-256 VÀ VI XỬ LÝ RISC-V TRÊN FPGA DE0-NANO

GVHD: TS. TRẦN QUANG NGUYÊN

Nhóm sinh viên thực hiện:

1. Đinh Khánh Đăng — MSSV: 23520224
2. Phạm Chí Đạt — MSSV: 23520265

Toàn bộ mã nguồn, testbench và tài liệu liên quan của đồ án được lưu tại GitHub:

https://github.com/DinhKhanhDang27/KLTN

ꕥ Tp. Hồ Chí Minh, 06/2026 ꕥ

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

……, ngày…….tháng……năm 20…

Người nhận xét

(Ký tên và ghi rõ họ tên)

BẢNG PHÂN CÔNG, ĐÁNH GIÁ THÀNH VIÊN:

Bảng 0.1: Bảng phân công, đánh giá thành viên

Họ và tên	MSSV	Phân công	Đánh giá
Đinh Khánh Đăng	23520224	-Thiết kế khối cầu giao tiếp sha256_avalon_wrapper.v để kết nối lõi băm với bus Avalon-MM (lập bản đồ thanh ghi Register Map, xử lý dữ liệu 32-bit ⇔ 512-bit). - Xây dựng môi trường mô phỏng tiền giám sát (Pre-simulation): Viết kịch bản testbench test_sha256_core.py/ test_riscv.py bằng Python (thư viện Cocotb) và file cấu hình Makefile. - Lập trình phần mềm điều khiển (mã nguồn Absemlly). - Chạy mô phỏng trên ModelSim, nạp mạch thực tế và sử dụng Signal Tap để kiểm thử xung tín hiệu.	100%
Phạm Chí Đạt	23520265	- Nghiên cứu thuật toán SHA-256 và thiết kế khối lõi phần cứng sha256_core.v bằng RTL Verilog. - Nghiên cứu RISCV datapath và thiết kế các khối phần cứng bằng RTL Verilog. - Thiết kế Máy trạng thái (FSM) gồm 64 vòng nén tính toán. - Cấu hình hệ thống SoC trong Platform Designer (Qsys): Tích hợp CPU RISCV, Interconnect Avalon, SDRAM Controller, On-chip Memory. - Thực hiện tổng hợp mạch (Synthesis), tối ưu hóa tài nguyên phần cứng, kiểm tra Timing (Slack) và Power Analyzer trên phần mềm Quartus Prime.	100%

Contents

DANH MỤC HÌNH ẢNH 6

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI 7

1. Đặt vấn đề và Tính cấp thiết của đề tài 7

2. Cơ sở lý thuyết về SHA-256 7

3. Cơ Sở Lý Thuyết RISCV 8

4. Tổng quan về SoC áp dụng trong đề tài 8

5. Thiết bị và Công cụ sử dụng 9

6. Lý do chọn FPGA DE0-Nano và bộ nhớ ngoài SDRAM 32MB 9

7. Phạm vi và Hạn chế của đề tài 10

CHƯƠNG 2. TỔNG QUAN THÀNH PHẦN THIẾT KẾ 12

1. Tổng quan về cấu trúc thiết kế 12

2. Giao tiếp Bus và Không gian Bộ nhớ (Bus Interface & Memory Map) 13

3. Chi tiết các khối chức năng 16

3.1 Vi xử lý RISCV 16

3.2 Hệ thống bộ nhớ (SDRAM Controller và On-chip Memory) 16

3.3 Khối giao tiếp JTAG UART 18

3.3 Khối Timer 19

3.4 Khối JTAG Avalon Master 19

3.5 Khối tăng tốc phần cứng SHA-256 19

CHƯƠNG 3: MÔ PHỎNG, KIỂM THỬ VÀ ĐÁNH GIÁ 25

I. Kiểm thử SHA-256 – Avalon wrapper IP. 25

1. Pre-Simulation: 25

2. Coverage: 29

3. Synthesis: 31

4. Implementation: 32

5. Post – Simulation: 33

II. Kiểm thử RISC-V 34

Pre-Simulation. 34

2.1. Cải tiến lõi RISC-V theo kiến trúc RV32I 5-stage pipeline 37

2.1.1. Mục tiêu cải tiến 37

2.1.2. Cấu trúc tổng thể của bộ mã nguồn pipeline 38

2.1.3. Luồng xử lý của kiến trúc 5-stage pipeline 40
2.1.4. Tập lệnh được hỗ trợ 41
2.1.5. Forwarding và Hazard Detection 41
2.1.6. Avalon-MM Load/Store Unit trong MEM stage 42
3.1. Kiểm thử và đánh giá lõi RISC-V RV32I pipeline 43
3.1.1. Phương pháp kiểm thử self-checking 43
3.1.2. Nhóm testcase hợp lệ 44
3.1.3. Nhóm testcase negative/trap 45
3.1.4. Coverage của các module RTL 46
3.1.5. Đo Fmax CPU-only bằng Quartus/TimeQuest 47
3.1.6. Đánh giá ý nghĩa của pipeline so với lõi chưa pipeline 48
III. Quy trình triển khai hệ thống 50
CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 54
1. Kết luận 54
2. Hạn chế 55
3. Hướng phát triển 55
TÀI LIỆU THAM KHẢO 56

DANH MỤC HÌNH ẢNH

Hình 1. 1 FPGA DE0-Nano thực tế 9
Hình 2. 1 Tổng quan hệ thống 12
Hình 2. 2 Hệ thống ban đầu đã kết nối tín hiệu 13
Hình 2. 3 Bus Avalon MM 14
Hình 2. 4 Địa chỉ nền của các khối thành phần 15
Hình 2. 5 Block Diagram của RISC-V 16
Hình 2. 6 On-chip Memory 17
Hình 2. 7 SDRAM Controller 18
Hình 2. 8 JTAG UART 18
Hình 2. 9 Timer 19
Hình 2. 10 Khối SHA256 20
Hình 2. 11 Tạo component IP cho SHA-256 20
Hình 3. 1 Các Ca Kiểm Thử Định Hướng (Directed Test Cases) 26
Hình 3. 2 Các Ca Kiểm Thử Ngẫu Nhiên (Random Test Cases) 27
Hình 3. 3 Các Ca Kiểm Thử Giao Thức (Protocol Test Cases) 28
Hình 3. 4 Coverage 29
Hình 3. 5 Synthesis 31
Hình 3. 6 Ước lượng tài nguyên 32
Hình 3. 7 Fmax và Critical Path 33
Hình 3. 8 Post functional synthesis 33
Hình 3. 9 Post Implementation timing 34
Hình 3. 10 Cấu trúc RISCV 35
Hình 3. 11 Presimulation RISC-V 36
Hình 3. 12 Fmax RISC-V 37
Hình 3. 13 Vị trí lõi RV32I pipeline trong hệ thống SoC RISC-V + Avalon-MM + SHA-256. 38
Hình 3. 14 Sơ đồ luồng dữ liệu và điều khiển của lõi RV32I 5-stage pipeline 40
Hình 3. 15 Mô hình kiểm thử chức năng cho lõi RV32I pipeline bằng Avalon RAM model. 44
Hình 3. 16 Kết quả kiểm thử legal RV32I pipeline trong QuestaSim/ModelSim. 45
Hình 3. 17 Kết quả kiểm thử các trường hợp illegal instruction và trap. 46
Hình 3. 18 Design Unit coverage của các module RTL trong Questa Coverage Report. 47
Hình 3. 19 Kết quả Fmax Summary của lõi RV32I pipeline trên Cyclone IV E. 48
Hình 3. 20 Tài nguyên phần cứng của thiết kế CPU-only pipeline sau Fitter. 48
Hình 3. 21 Sau khi chạy lệnh tạo tcl 51
Hình 3. 22 Sau khi refresh connection 52
Hình 3. 23 Kết quả sau khi Hash 53
Hình 3. 24 Kết quả sau khi hash soft 53
Hình 3. 25 Kết quả sau khi hash IP core 53

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI

1. Đặt vấn đề và Tính cấp thiết của đề tài

Trong kỷ nguyên của Internet of Things (IoT) và Blockchain, an toàn thông tin đã trở thành một yếu tố tiên quyết. Thuật toán băm mật mã SHA-256 là một trong những giải pháp nền tảng, được ứng dụng rộng rãi để đảm bảo tính toàn vẹn dữ liệu và xác thực chữ ký số. Tuy nhiên, việc thực thi SHA-256 hoàn toàn bằng phần mềm trên các bộ vi xử lý đa dụng, điển hình là các lõi CPU RISC-V, gặp phải thách thức lớn về hiệu năng. Do đòi hỏi các phép toán logic phức tạp lặp đi lặp lại trên những khối dữ liệu lớn, thuật toán sẽ chiếm dụng phần lớn chu kỳ xử lý, gây ra độ trễ đáng kể và làm giảm khả năng đáp ứng của CPU cho các tác vụ khác.

Để giải quyết bài toán này, mô hình đồng thiết kế phần cứng/phần mềm (Hardware/Software Co-design) được xem là giải pháp tối ưu. Thay vì để CPU tự vật lộn với gánh nặng tính toán, phương pháp này đề xuất hiện thực hóa lõi SHA-256 thành một khối tăng tốc phần cứng (Hardware Accelerator) chuyên dụng trên FPGA.

Trong kiến trúc hệ thống trên chip (SoC) này, vi xử lý RISC-V sẽ đóng vai trò là bộ não điều khiển trung tâm. Thay vì trực tiếp tính toán băm, CPU RISC-V chỉ cần thực hiện các tác vụ quản lý ở mức cao như: gửi dữ liệu đầu vào cho khối tăng tốc, ra lệnh bắt đầu, và đọc kết quả sau khi hoàn thành. Việc phân chia vai trò này giúp giải phóng hoàn toàn tài nguyên cho CPU, cho phép nó xử lý song song các công việc khác, từ đó tối ưu hóa hiệu suất toàn hệ thống.

Vì vậy, nhóm quyết định thực hiện đề tài: "Thiết kế và xây dựng Hệ thống trên chip (SoC) tích hợp bộ tăng tốc phần cứng SHA-256 và vi xử lý RISC-V trên FPGA" nhằm hiện thực hóa một giải pháp kiến trúc tiên tiến, đáp ứng nhu cầu về hiệu năng và bảo mật trong các ứng dụng nhúng hiện đại.

2. Cơ sở lý thuyết về SHA-256

SHA-256 (Secure Hash Algorithm 256-bit) là hàm băm mật mã học được thiết kế nhằm chuyển đổi một dữ liệu đầu vào có kích thước bất kỳ thành một chuỗi kết quả có độ dài cố định là 256 bit. Thuật toán sở hữu ba đặc tính bảo mật cốt lõi bao gồm: tính tất định, tính một chiều, và hiệu ứng thác.

Về nguyên lý hoạt động, quá trình băm của SHA-256 trải qua hai giai đoạn chính:

- **Giai đoạn tiền xử lý:** Dữ liệu đầu vào được thêm các bit đệm bắt buộc sao cho tổng kích thước của thông điệp trở thành bội số của 512 bit.
- **Giai đoạn tính toán lặp:** Thông điệp được chia nhỏ thành các khối (block) kích thước 512 bit. Mỗi khối dữ liệu sẽ trải qua một chu trình tính toán gồm 64 vòng lặp tuần tự, sử dụng kết hợp các phép toán logic cơ bản như dịch bit (shift), xoay bit (rotate) và các phép toán Boolean (AND, XOR, NOT). Kết quả băm của khối trước

sẽ được dùng làm trạng thái đầu vào để tính toán cho khối kế tiếp, lặp lại liên tục cho đến khối cuối cùng để xuất ra giá trị băm 256 bit hoàn chỉnh.

3. Cơ Sở Lý Thuyết RISCV

RISC-V là một kiến trúc tập lệnh (Instruction Set Architecture - ISA) mở và miễn phí, được phát triển tại Đại học California, Berkeley. Khác với các kiến trúc thương mại như ARM hay x86, tính "mở" của RISC-V cho phép bất kỳ ai cũng có thể tự do thiết kế, hiện thực hóa và mở rộng các bộ vi xử lý mà không cần trả phí bản quyền. Nền tảng của RISC-V tuân thủ triết lý Máy tính với tập lệnh đơn giản hóa (RISC), tập trung vào việc xây dựng một tập lệnh nhỏ gọn, hiệu quả và có tính module hóa cao. Điều này cho phép tùy biến các lõi CPU cho nhiều mục đích khác nhau, từ các hệ thống nhúng công suất thấp đến các siêu máy tính hiệu năng cao.

Lõi rtl_riscv trong đồ án được xây dựng dựa trên tập lệnh số nguyên cơ sở 64-bit, ký hiệu là RV64I. Kiến trúc này tuân thủ nghiêm ngặt nguyên tắc "load-store", nghĩa là các lệnh tính toán (như cộng, trừ, logic) chỉ có thể thao tác trên dữ liệu chứa trong các thanh ghi. Để xử lý dữ liệu từ bộ nhớ, CPU phải dùng lệnh load để tải dữ liệu vào thanh ghi, và lệnh store để ghi kết quả từ thanh ghi trở lại bộ nhớ. Lõi RV64I cung cấp 32 thanh ghi đa dụng 64-bit (từ x0 đến x31), trong đó thanh ghi x0 luôn được giữ cứng ở giá trị 0. Hầu hết các lệnh đều có định dạng dài 32-bit cố định, giúp đơn giản hóa đáng kể quá trình giải mã lệnh trong một thiết kế single-cycle.

Trong kiến trúc của đồ án, lõi rtl_riscv không hoạt động độc lập mà được tích hợp vào một Hệ thống trên chip (SoC) lớn hơn thông qua khối riscv_avalon_wrapper. Khối wrapper này trang bị cho CPU một giao diện bus Avalon-MM (Memory-Mapped), biến nó thành một master điều khiển trung tâm. Thông qua giao diện này, vi xử lý RISC-V có thể thực thi firmware để giao tiếp và điều phối hoạt động của các ngoại vi khác trong hệ thống, chẳng hạn như gửi dữ liệu cho bộ tăng tốc SHA-256, đọc/ghi vào bộ nhớ SDRAM, và xuất dữ liệu qua UART. Việc điều khiển được thực hiện bằng các lệnh load/store tới các địa chỉ đã được ánh xạ của từng ngoại vi, thể hiện rõ vai trò của CPU trong một hệ thống đồng thiết kế phần cứng/phần mềm.

4. Tổng quan về SoC áp dụng trong đề tài

Kiến trúc SoC áp dụng trong đề tài được xây dựng dựa trên nguyên lý kết nối các thành phần IP thông qua một mạng giao tiếp bus chuẩn hóa. Hệ thống bao gồm một bộ vi xử lý điều khiển trung tâm phối hợp cùng cấu trúc bộ nhớ phân cấp và bộ tăng tốc phần cứng thông qua các thành phần cốt lõi:

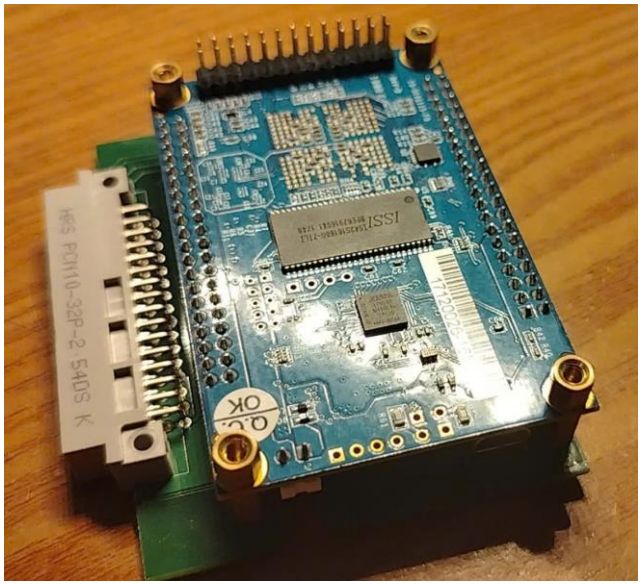
- **Chuẩn giao tiếp Bus Avalon Memory-Mapped (Avalon-MM):** Hệ thống sử dụng chuẩn giao thức Avalon-MM để kết nối giữa vi xử lý (Master) và các ngoại vi (Slave). Cơ chế này hoạt động dựa trên nguyên lý ánh xạ thanh ghi (Register Mapping), cho phép CPU truy xuất và cấu hình dữ liệu vào bộ tăng tốc phần cứng một cách đồng bộ thông qua các địa chỉ ô nhớ được định nghĩa trước.

- **Hệ thống bộ nhớ phân cấp:** Hệ thống tích hợp On-chip Memory (32 KB) làm vùng nhớ nội tốc độ cao bên trong FPGA để chứa mã lệnh khởi động hệ thống. Đồng thời, bộ điều khiển SDRAM Controller được cấu hình để quản lý bộ nhớ ngoài 32 MB SDRAM, cung cấp không gian lưu trữ dung lượng lớn cho các tệp tin và luồng dữ liệu cần thực hiện băm mật mã.
- **Vi xử lý nhúng RISC-V**: Là một lõi vi xử lý 64-bit được thiết kế dựa trên kiến trúc tập lệnh (ISA) mở RISC-V và được cài đặt trên tài nguyên FPGA. Trong hệ thống, lõi RISC-V đóng vai trò là Master điều khiển chính, thực thi chương trình firmware (mã Assembly) để quản lý luồng dữ liệu, cấu hình các thanh ghi chức năng và kiểm soát trạng thái của bộ tăng tốc phần cứng.

5. Thiết bị và Công cụ sử dụng

Quá trình hoàn thiện đồ án đòi hỏi sự phối hợp chặt chẽ giữa thiết bị phần cứng vật lý và hệ thống công cụ phần mềm chuyên dụng:

- **Thiết bị phần cứng:** Sử dụng bo mạch phát triển **Terasic DE0-Nano** tích hợp dòng chip FPGA Altera Cyclone IV (EP4CE22F17C6N).
- **Công cụ thiết kế hệ thống:** Sử dụng phần mềm **Quartus Prime** để tổng hợp mạch, phân tích tài nguyên phần cứng và tối ưu hóa thiết kế RTL. Công cụ **Qsys (Platform Designer)** được dùng để thiết kế, kết nối bus Avalon và tích hợp các khối thành phần thành một hệ thống SoC hoàn chỉnh.
- **Công cụ kiểm chứng:** Quy trình kiểm tra thiết kế được thực hiện toàn diện qua hai bước. Đầu tiên là quá trình mô phỏng chức năng sử dụng phần mềm **ModelSim** kết hợp với framework **CocoTB (Python)** để kiểm tra tính đúng đắn của phần cứng trước khi mạch nạp.



Hình 1. 1 FPGA DE0-Nano thực tế

6. Lý do chọn FPGA DE0-Nano và bộ nhớ ngoài SDRAM 32MB

Tài nguyên Logic vừa vặn cho kiến trúc SoC quy mô vừa và nhỏ: Chip Cyclone IV trên DE0-Nano cung cấp 22.320 phần tử logic và 594 Kbits bộ nhớ nhúng (On-chip Memory). Lượng tài nguyên này cấu hình vừa đủ cho một hệ thống SoC hoàn chỉnh và vẫn còn thừa đủ không gian mạch logic để triển khai lõi phần cứng của bộ tăng tốc SHA-256.

Bắt buộc phải có SDRAM 32MB để xử lý dữ liệu lớn: Bộ nhớ nội (On-chip Memory) tích hợp sẵn trong chip FPGA vô cùng hạn chế (chỉ vài chục KB). Trong khi đó, thuật toán SHA-256 thường được dùng để băm các luồng dữ liệu hoặc các tệp tin có kích thước lớn. Việc DE0-Nano trang bị sẵn chip nhớ ngoài SDRAM 32MB là yếu tố quyết định, đóng vai trò làm bộ nhớ phân cấp giúp CPU Nios II nạp toàn bộ chương trình thực thi mã C và lưu trữ các mảng dữ liệu lớn trước khi đẩy tuần tự xuống bộ tăng tốc phần cứng.

Đồng bộ xung nhịp (Clock) hệ thống ổn định: Bo mạch tích hợp sẵn thạch anh tạo xung nhịp 50 MHz. Tần số này là mức chuẩn lý tưởng để chạy vi xử lý Nios II ổn định, đồng thời giúp việc truyền nhận dữ liệu qua Bus Avalon-MM giữa CPU, bộ điều khiển SDRAM và bộ tăng tốc phần cứng SHA-256 diễn ra chính xác theo thời gian thực mà không gặp các lỗi về bóp méo thời gian.

7. Phạm vi và Hạn chế của đề tài

Phạm vi nghiên cứu: Đề tài tập trung thiết kế và thực nghiệm bộ tăng tốc phần cứng cho thuật toán SHA-256 trong môi trường SoC nhúng, sử dụng tài nguyên phần cứng hiện có trên bo mạch DE0-Nano.

Hạn chế của đề tài:

Giới hạn về Kiến trúc và Tập lệnh của Lõi RISC-V

Lõi vi xử lý RISC-V được phát triển trong khuôn khổ đề tài là một thiết kế 64-bit tùy chỉnh, tập trung vào việc thực thi một tập lệnh cơ bản (khả năng là RV64I). Do giới hạn về thời gian và phạm vi, lõi xử lý này không cài đặt các phần mở rộng tiêu chuẩn như M (nhân/chia số nguyên), A (lệnh nguyên tử), F/D (tính toán dấu phẩy động) hay C (lệnh nén). Thêm vào đó, các tính năng nâng cao như cơ chế xử lý ngắt ngoại lệ (trap/exception) cho các lệnh không hợp lệ, hệ thống phân cấp đặc quyền (privilege modes), và bộ quản lý bộ nhớ (MMU) cũng chưa được tích hợp. Điều này định hướng vai trò của lõi RISC-V như một bộ điều khiển nhúng chuyên dụng cho các tác vụ cụ thể thay vì một CPU đa dụng cho các hệ điều hành phức tạp.

Giới hạn trong Môi trường Xác minh (Verification)

Mặc dù môi trường xác minh cho các khối IP đã được xây dựng tương đối hoàn chỉnh với các thành phần như mô hình tham chiếu (golden model), bảng điểm (scoreboard) và các kịch bản kiểm thử, quá trình xác minh vẫn tồn tại một số giới hạn. Cụ thể, độ bao phủ mã (code coverage) cho lõi RISC-V chưa đạt 100% ở các hạng mục như bao phủ điều kiện (condition coverage) và bao phủ chuyển đổi (toggle coverage). Nguyên nhân chủ yếu là do các chương trình firmware kiểm thử chỉ hoạt động trên các giá trị dữ liệu và không gian địa chỉ nhỏ, dẫn đến việc các bit bậc cao của bus dữ liệu, bus địa chỉ và thanh ghi 64-bit không được thực thi đầy đủ. Một số nhánh mã liên quan đến các trường hợp lỗi hoặc các đường giải mã không hợp lệ trong thiết kế đã được ghi nhận là không thể tiếp cận (unreachable) và cần được miễn trừ (waive) thay vì tạo stimulus để kích hoạt.

Giới hạn của Khối tăng tốc SHA-256

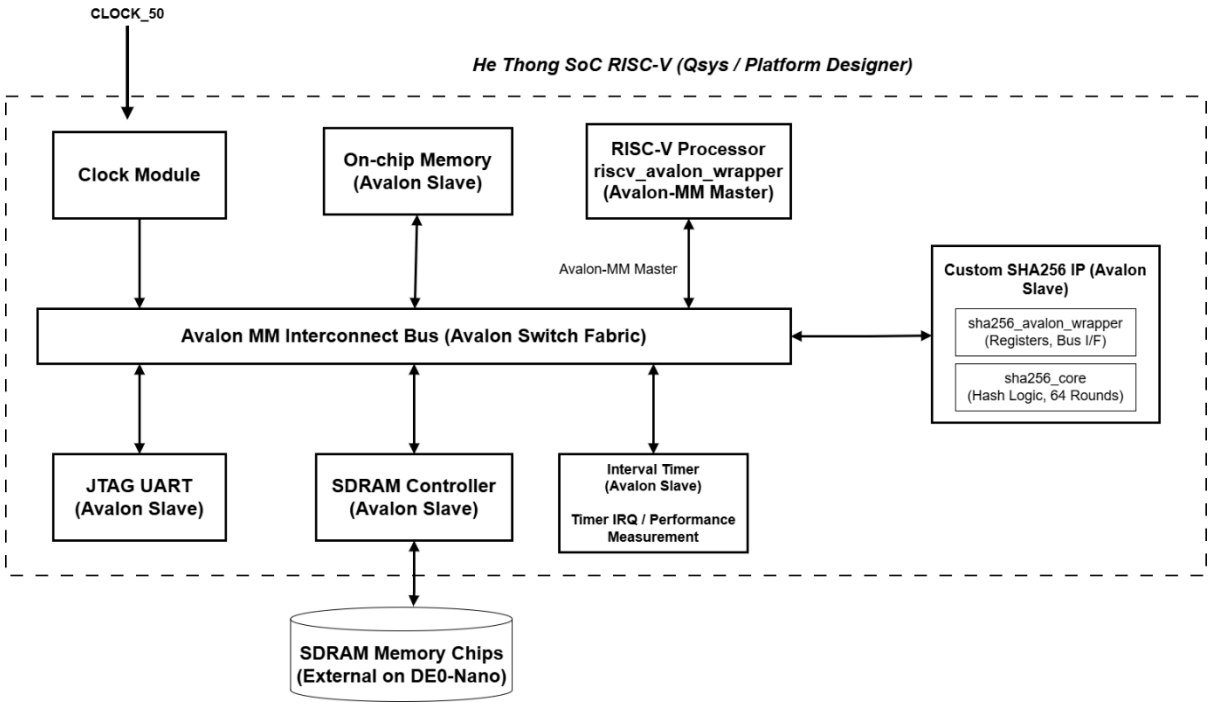
Khối IP tăng tốc SHA-256 được thiết kế để xử lý từng khối tin nhắn 512-bit độc lập. Thiết kế này chưa hỗ trợ xử lý các tin nhắn có độ dài lớn hơn một khối một cách tự động trong phần cứng. Quá trình đệm tin nhắn (message padding) theo chuẩn SHA-256 không được thực hiện bởi phần cứng mà phải được chuẩn bị trước bởi firmware chạy trên CPU RISC-V. Do đó, CPU phải chịu trách nhiệm chia tin nhắn dài thành các khối 512-bit, thực hiện padding cho khối cuối cùng, và gọi khối tăng tốc lặp lại nhiều lần. Điều này làm tăng độ phức tạp của phần mềm và có thể ảnh hưởng đến hiệu năng tổng thể của hệ thống khi xử lý các luồng dữ liệu lớn.

Giới hạn về Tích hợp Hệ thống trên SoC

Hệ thống trên chip (SoC) được xây dựng dựa trên các thành phần IP kết nối qua bus Avalon, và được triển khai trên kit FPGA DE0-Nano. Việc sử dụng kit FPGA này đi kèm với giới hạn về tài nguyên logic và bộ nhớ có sẵn. Mặc dù hệ thống có sử dụng SDRAM ngoài, giao tiếp giữa CPU và bộ nhớ ngoài được thực hiện thông qua một bộ điều khiển SDRAM và bus hệ thống, có thể tạo ra độ trễ truy cập đáng kể so với bộ nhớ on-chip. Hơn nữa, các thành phần như bộ điều khiển truy cập bộ nhớ trực tiếp (DMA) đã được đề cập trong kế hoạch nhưng có thể chưa được tích hợp hoàn chỉnh, khiến cho việc truyền dữ liệu lớn giữa bộ nhớ và khối SHA-256 vẫn phụ thuộc vào sự can thiệp của CPU, làm giảm hiệu quả xử lý song song.

CHƯƠNG 2. TỔNG QUAN THÀNH PHẦN THIẾT KẾ

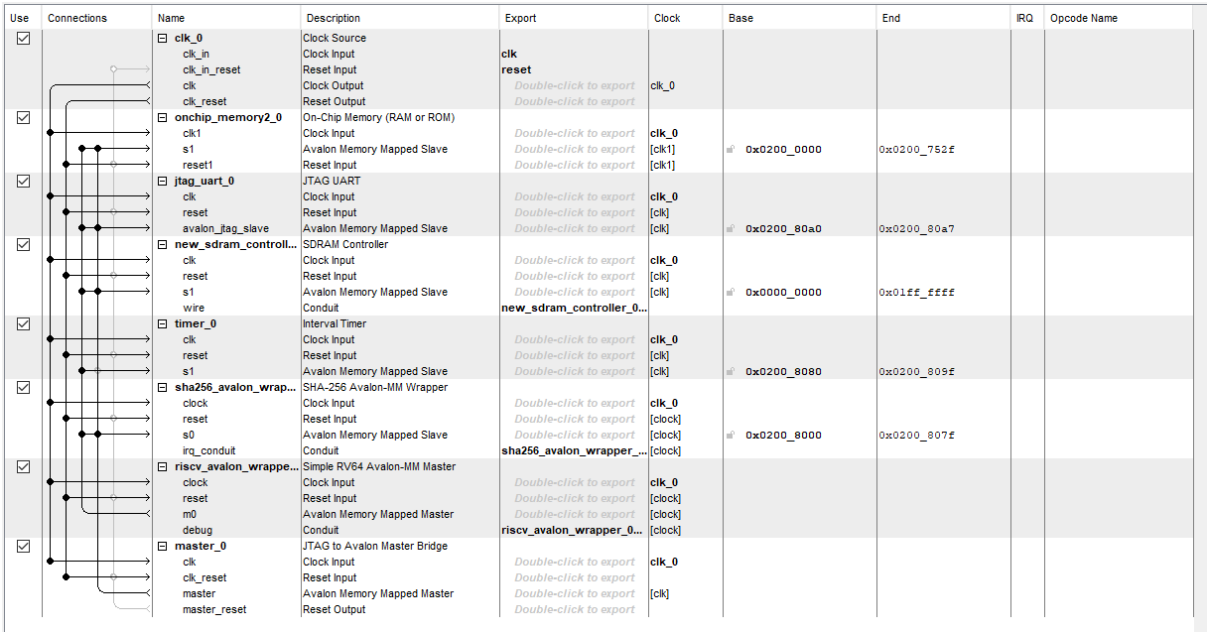
1. Tổng quan về cấu trúc thiết kế



Hình 2. 1 Tổng quan hệ thống

Hệ thống được thiết kế theo kiến trúc SoC dựa trên vi xử lý RISC-V, triển khai trên nền tảng FPGA Cyclone IV E (bo mạch DE0-Nano) và tích hợp qua công cụ Qsys (Platform Designer) của Intel Quartus.

- **Ở mức đỉnh (Top-level):** Hệ thống tiếp nhận nguồn xung nhịp cấu hình CLOCK_50 có tần số 50 MHz từ thạch anh onboard để phân phối cho toàn mạch. Module Top đóng vai trò ánh xạ và kết nối trực tiếp các chân giao tiếp ngoại vi vật lý trên bo mạch vào khối SoC trung tâm.
- **Kiến trúc nội tại SoC:** Bao gồm các thành phần cốt lõi kết nối đồng bộ qua hệ thống Bus chuẩn hóa: vi xử lý mềm RISC-V, bộ nhớ nội tốc độ cao (On-chip Memory), bộ điều khiển bộ nhớ ngoài (SDRAM Controller), khối giao tiếp nạp/gỡ lỗi JTAG UART, và bộ tăng tốc phần cứng chuyên dụng cho thuật toán băm SHA-256 do nhóm tự thiết kế.

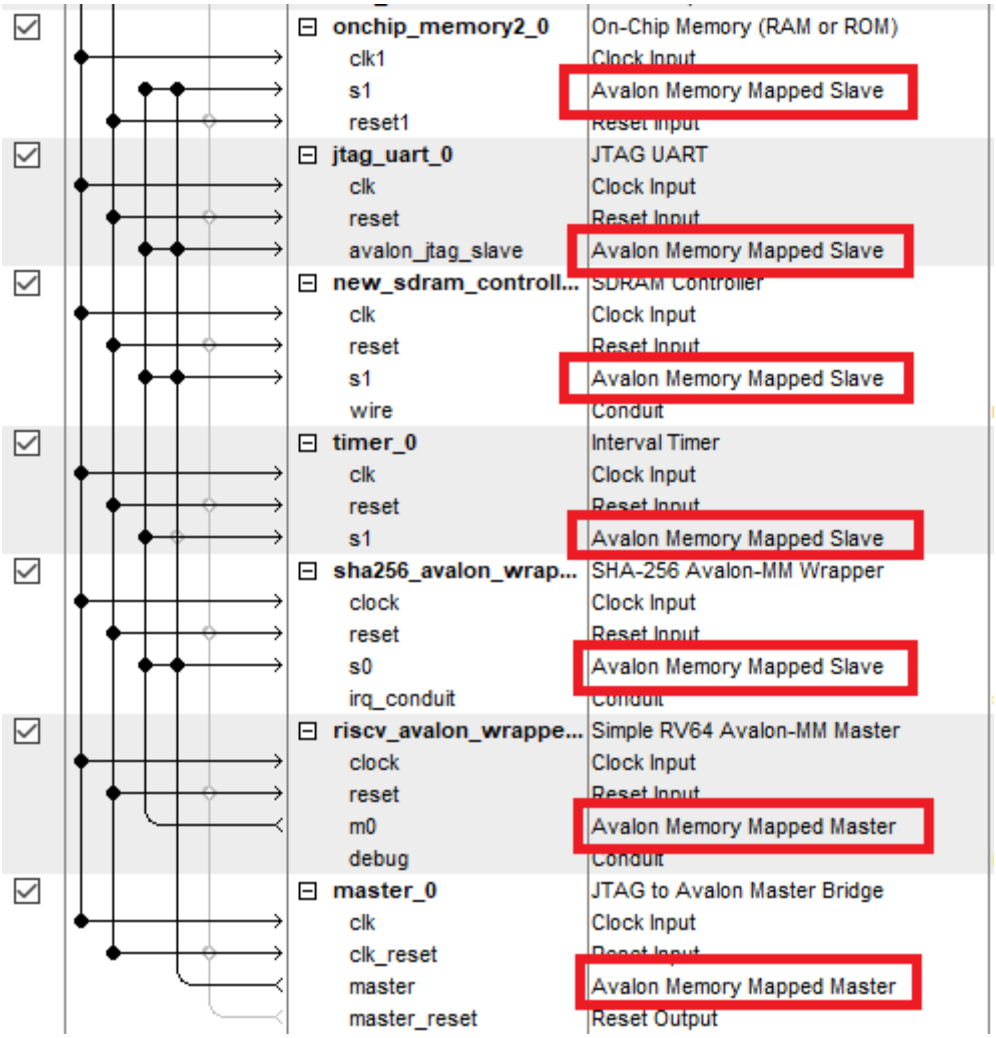


Hình 2. 2 Hệ thống ban đầu đã kết nối tín hiệu

2. Giao tiếp Bus và Không gian Bộ nhớ (Bus Interface & Memory Map)

Hệ thống sử dụng chuẩn giao thức giao tiếp Avalon Memory-Mapped (Avalon-MM) để liên kết đồng bộ các thành phần IP core. Trong kiến trúc này, vi xử lý RISCV đóng vai trò là Master điều phối toàn bộ Bus thông qua hai cổng giao tiếp độc lập:

- **Instruction Master Port:** Chuyên trách tác vụ nạp mã lệnh (instruction fetch) từ các vùng nhớ chương trình như **On-chip Memory** hoặc **SDRAM**. Cổng này đảm bảo CPU luôn có luồng lệnh thực thi liên tục mà không bị xung đột với luồng dữ liệu.
- **Data Master Port:** Phục vụ toàn bộ quá trình đọc/ghi dữ liệu thực tế. Cổng này chịu trách nhiệm truy xuất các mảng dữ liệu lớn chứa trong **SDRAM** để trung chuyển, đồng thời thực hiện ánh xạ và cấu hình trực tiếp xuống các thanh ghi điều khiển, thanh ghi trạng thái của ngoại vi **JTAG UART** và **bộ tăng tốc phần cứng SHA-256 (Slave)**.

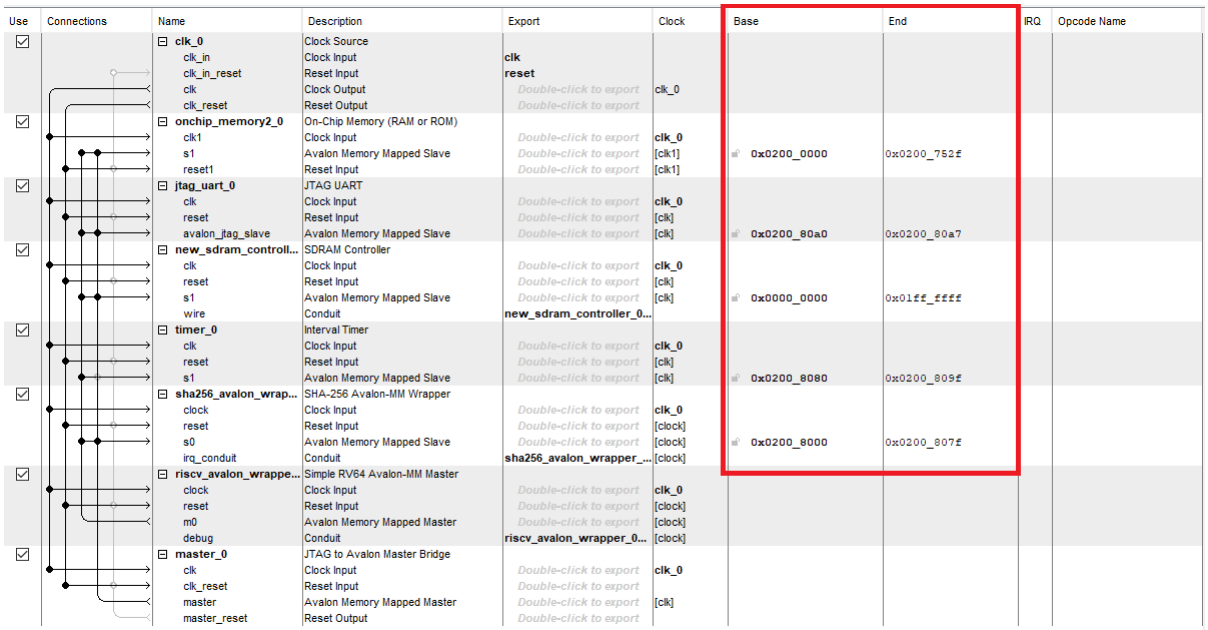


Hình 2. 3 Bus Avalon MM

Các khối ngoại vi trong hệ thống đóng vai trò là các Slave trên bus Avalon-MM và được ánh xạ vào không gian địa chỉ bộ nhớ của bộ xử lý RISC-V theo cơ chế Memory-mapped I/O. Việc truy xuất các khối được quản lý thông qua bảng địa chỉ cơ sở như sau:

- SDRAM Controller: 0x00000000 (Không gian quản lý 32 MB).
- On-chip Memory: 0x02000000 (Không gian quản lý 30,000 Bytes, khoảng 29.3 KB).
- SHA-256 IP: 0x02008000 (Không gian quản lý 128 Bytes).
- Timer: 0x02008080 (Không gian quản lý 32 Bytes).
- JTAG UART: 0x020080A0 (Không gian quản lý 8 Bytes).

Ngoài ra, hệ thống còn có master_0 là JTAG Avalon Master dùng để truy cập/debug hệ thống qua JTAG, không phải là một Slave ngoại vi trong bảng ánh xạ bộ nhớ của CPU.



Hình 2. 4 Địa chỉ nền của các khối thành phần

Sự khác biệt lớn nhất trong giao tiếp Avalon-MM của hệ thống là khả năng đáp ứng linh hoạt nhờ cơ chế chuyển đổi giao thức (Protocol Adaptation) của tầng liên kết Avalon Interconnect. Trong khi bộ điều khiển SDRAM đòi hỏi các tính năng phức tạp để tối ưu băng thông (waitrequest, burst, byteenable), thì lớp bọc (Wrapper) của khối SHA-256 lại là một Slave tối giản dạng tập thanh ghi (Register File) 32-bit đáp ứng tức thời và không có độ trễ.

Sự bất đối xứng giao thức này được xử lý hoàn hảo thông qua 3 cơ chế:

Phản hồi tức thời: Lớp bọc SHA-256 luôn sẵn sàng dữ liệu ngay trong chu kỳ read/write. Hệ thống tự động giải phóng mạch trọng tài mà không cần chèn chu kỳ đợi, giúp CPU Nios II cấu hình thanh ghi lệnh và dữ liệu với độ trễ tiệm cận bằng 0.

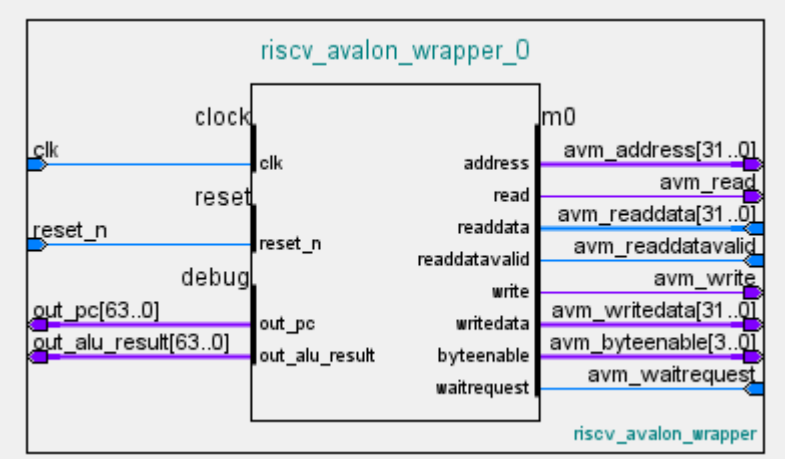
Thích ứng độ rộng dữ liệu: Dù lõi SHA-256 quản lý thanh ghi 32-bit, hệ thống vẫn hỗ trợ CPU ghi phân đoạn (8-bit hoặc 16-bit) nhờ tín hiệu byteenable[3:0]. Tầng liên kết tự động tạo mặt nạ byte, giúp Wrapper chỉ cập nhật dữ liệu vào byte cần thiết mà không làm ảnh hưởng vùng lân cận.

Cô lập phân vùng: Bằng cách tách biệt luồng xử lý phức tạp của SDRAM (vốn hay hoãn bus bằng waitrequest để làm tươi ô nhớ) khỏi luồng của SHA-256, Avalon Fabric đảm bảo trạng thái trễ của bộ nhớ ngoài không bị lan truyền làm ảnh hưởng đến tính thời gian thực của bộ tăng tốc.

3. Chi tiết các khối chức năng

3.1 Vi xử lý RISCV

Trong dự án này, bộ xử lý trung tâm không sử dụng Nios II mà được thiết kế dưới dạng lõi RISC-V tùy chỉnh, cụ thể là khối riscv_avalon_wrapper_0. Lõi xử lý hoạt động với xung nhịp 50 MHz, được cấp từ clk_0 trong hệ thống Platform Designer. Kiến trúc bên trong sử dụng thanh ghi, PC và ALU 64-bit theo hướng RV64 đơn giản, trong khi giao tiếp với các ngoại vi được thực hiện thông qua bus Avalon-MM Master 32-bit. Thiết kế không tích hợp bộ nhớ đệm lệnh và dữ liệu; lệnh được lấy từ khối instruction_memory, còn các thao tác đọc/ghi dữ liệu được truy cập trực tiếp qua không gian Memory-mapped I/O. Bus địa chỉ Avalon-MM có độ rộng 32-bit, bus dữ liệu đọc/ghi có độ rộng 32-bit, phù hợp để truy xuất SDRAM, On-chip Memory, Timer, JTAG UART và khối SHA-256 IP trong hệ thống

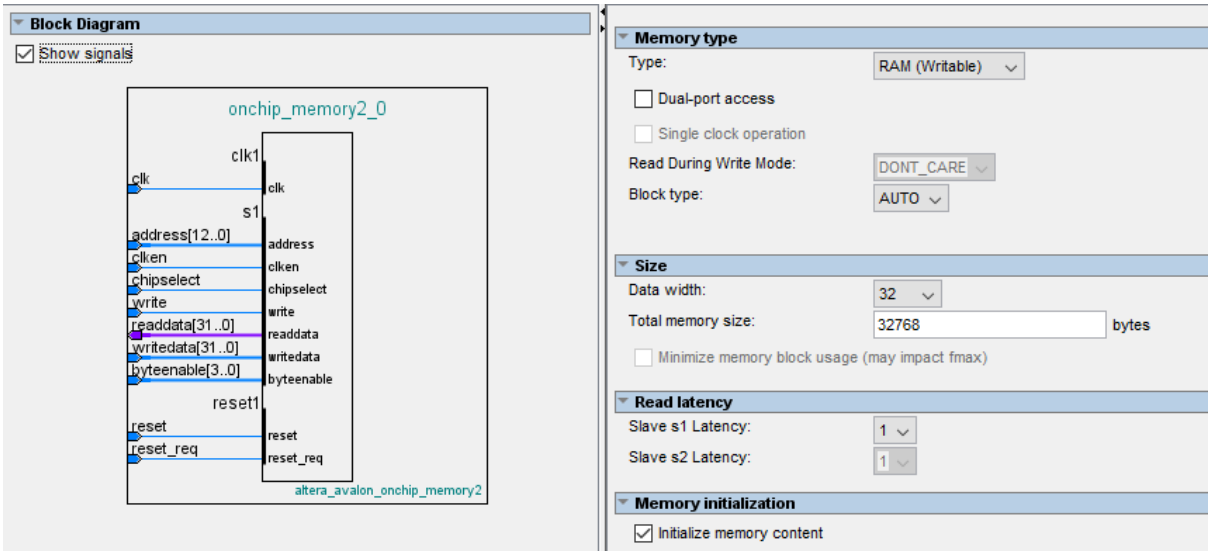


Hình 2. 5 Block Diagram của RISC-V

3.2 Hệ thống bộ nhớ (SDRAM Controller và On-chip Memory)

On-Chip Memory

Có dung lượng 32 KB, vận hành với bus dữ liệu 32-bit. Khối nhớ này có độ trễ cực thấp, chứa mã lệnh khởi động (reset/exception vector) và bộ chương trình điều khiển cơ bản.



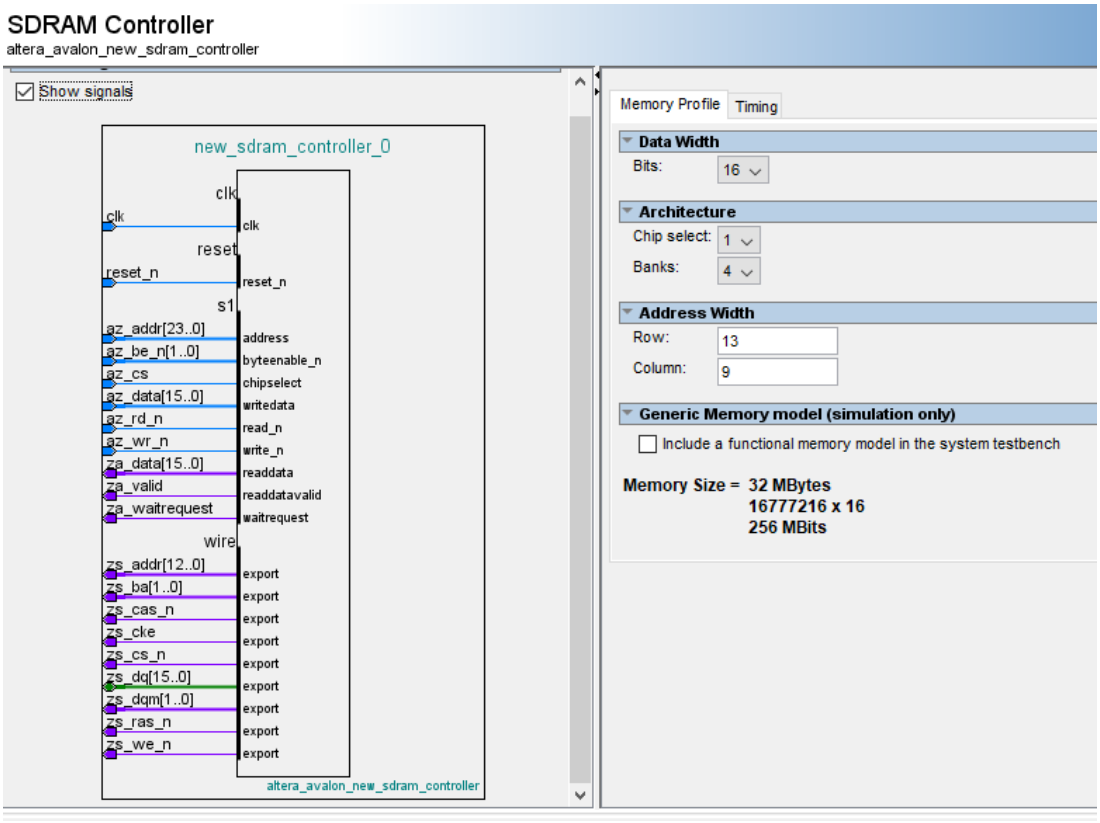
Hình 2. 6 On-chip Memory

SDRAM Controller

Bộ ngoại vi SDRAM được cấu hình tối ưu tài nguyên vật lý trên bo mạch DE0-Nano với các thông số cấu trúc cốt lõi sau:

- **Độ rộng dữ liệu vật lý 16-bit:** Bắt nguồn từ đường bus chân mạch DRAM_DQ[15:0]. Khi CPU Nios II (kiến trúc 32-bit) truy xuất bộ nhớ, tầng liên kết Avalon Interconnect/Adapter sẽ tự động thực hiện cơ chế tách/ghép một chu kỳ 32-bit thành hai truy cập 16-bit liên tiếp xuống SDRAM Controller.
- **Cấu trúc ma trận ô nhớ và Bank:** Chip nhớ sử dụng duy nhất 1 Chip Select (CS = 1), tổ chức thành 4 Banks độc lập (2-bit địa chỉ Bank), ma trận gồm 13-bit địa chỉ hàng (Row) và 9-bit địa chỉ cột (Column).
- **Định tuyến không gian nhớ:** Tổng số bit địa chỉ nội bộ là 24-bit (13 Row + 9 Column + 2 Bank), tạo ra không gian lưu trữ 2^{24} = 16.777.216 từ mã (words). Với kích thước mỗi word là 16-bit (2 bytes), tổng dung lượng khả dụng đạt đúng: 16.777.216 * 2 bytes = 32 MB.

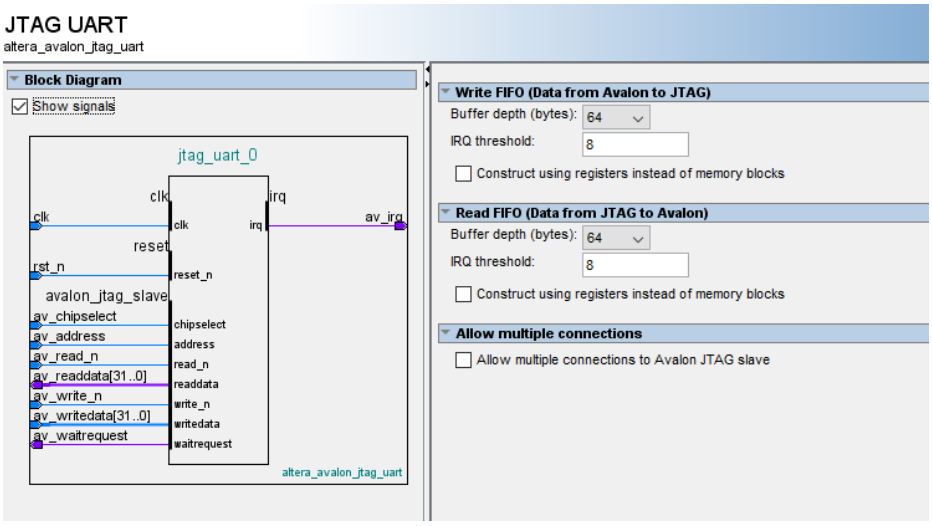
Cấu trúc vật lý này quyết định việc Qsys ánh xạ chính xác toàn bộ 32 MB dung lượng vào không gian bộ nhớ Avalon-MM tại địa chỉ cơ sở 0x0200_0000 với kích thước vùng nhớ (Span) là 33554432, như được tự động định nghĩa trong file tiêu đề phần mềm system.h.



Hình 2. 7 SDRAM Controller

3.3 Khối giao tiếp JTAG UART

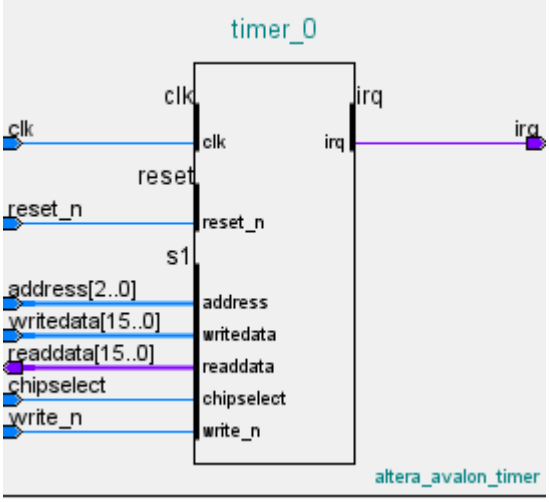
Khối giao tiếp này chiếm dụng 8 bytes không gian bộ nhớ, tích hợp sẵn hai bộ đệm FIFO (đọc và ghi) có dung lượng 64 bytes và sử dụng ngắt (IRQ 0). Nó tạo ra kênh truyền thông chuẩn nối tiếp giữa Nios II và máy tính host (thông qua JTAG), cho phép phần mềm C dùng lệnh printf để xuất log trạng thái và chuỗi mã băm kết quả trực tiếp ra màn hình Console.



Hình 2. 8 JTAG UART

3.3 Khối Timer

Khối timer_0 là bộ định thời Avalon Timer được tích hợp trong hệ thống Qsys, hoạt động với xung nhịp 50 MHz từ clk_0. Timer này có bộ đếm 32-bit, chu kỳ cấu hình 1 ms và hỗ trợ chức năng snapshot để đọc giá trị bộ đếm tại thời điểm cần đo. Trong không gian địa chỉ Memory-mapped I/O, timer_0 được ánh xạ tại địa chỉ cơ sở 0x02008080 với vùng quản lý 32 Bytes. Khối này có thể được bộ xử lý RISC-V truy cập thông qua bus Avalon-MM để phục vụ các chức năng đo thời gian, tạo trễ hoặc kiểm tra hiệu năng xử lý.



Hình 2. 9 Timer

3.4 Khối JTAG Avalon Master

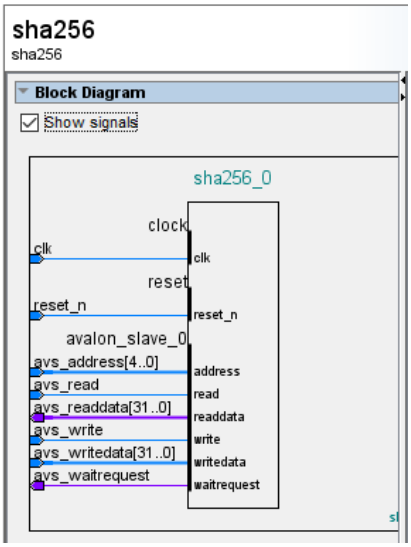
Khối master_0 là JTAG Avalon Master được thêm vào hệ thống nhằm hỗ trợ truy cập và kiểm tra các Slave Avalon-MM thông qua giao tiếp JTAG. Khác với các ngoại vi như Timer, JTAG UART hay SHA-256 IP, master_0 đóng vai trò là Master trên bus nên không được ánh xạ như một vùng nhớ Slave riêng. Khối này có thể truy xuất trực tiếp đến các vùng địa chỉ của hệ thống như SDRAM, On-chip Memory, SHA-256 IP và JTAG UART, giúp thuận tiện cho việc debug, nạp dữ liệu, quan sát thanh ghi và kiểm tra hoạt động của hệ thống trong quá trình phát triển trên FPGA.

3.5 Khối tăng tốc phần cứng SHA-256

Khối Custom IP được chia làm hai phần kiến trúc rõ rệt để thực thi tác vụ tăng tốc phần cứng:

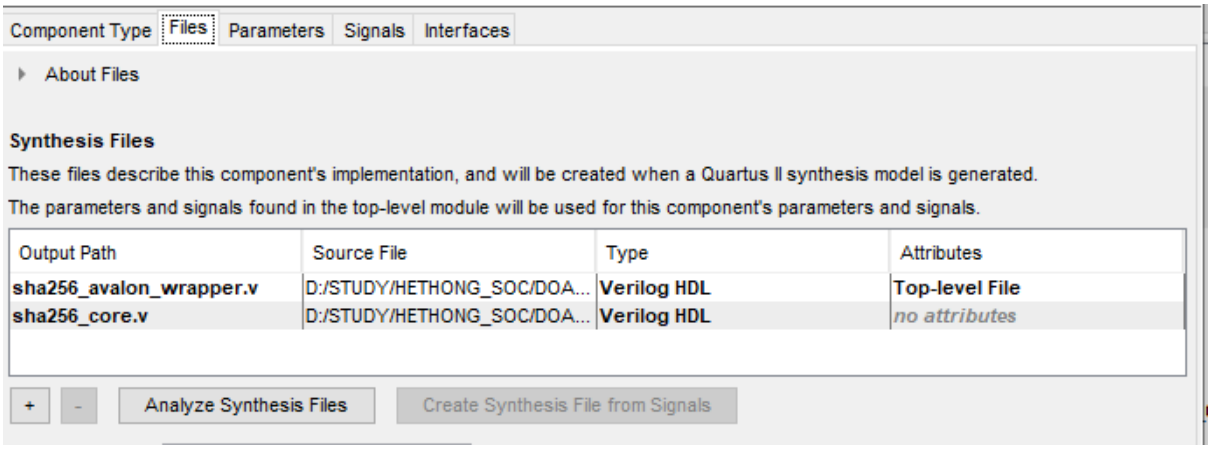
- **Lớp giao tiếp Avalon-MM Wrapper (sha256_avalon_wrapper):** Đóng vai trò cầu nối, quản lý tập thanh ghi 32-bit ánh xạ bộ nhớ để giao tiếp với CPU Nios II. Trong đó, các thanh ghi 0–15 (512-bit) chứa khối dữ liệu đầu vào; thanh ghi 16 nhận lệnh điều khiển (start/init) từ CPU để kích xung khởi động lõi; thanh ghi 17 phục vụ cơ chế thăm dò trạng thái (polling cờ ready/hash_valid); và các thanh ghi 18–25 lưu trữ kết quả băm 256-bit hoàn chỉnh.

- **Lõi xử lý SHA-256 (sha256_core):** Nhận trực tiếp xung nhịp 50 MHz cùng luồng điều khiển và khối dữ liệu 512-bit song song từ Wrapper. Lõi tích hợp một máy trạng thái hữu hạn (FSM) gồm 3 trạng thái (IDLE, CALC, DONE) để đồng bộ chu trình nén lặp qua 64 vòng (rounds 0–63), sử dụng 64 hằng số chuẩn K_t, mảng bộ nhớ mở rộng thông điệp 64 từ w_mem và 8 thanh ghi trạng thái (a đến h). Sau khi hoàn thành tính toán, lõi dựng cờ hash_valid và trả kết quả 256-bit về lớp Wrapper.



Hình 2. 10 Khối SHA256

RTL của SHA-256 sẽ bao gồm 2 khối là **sha256 core** và **sha256_avalon_wrapper,** với top module là khối wrapper:



Hình 2. 11 Tạo component IP cho SHA-256

Sha256 Avalon Wrapper

Lõi sha256_core là một IP phần cứng thô (Raw IP), tính toán dữ liệu dạng khối lớn song song (512-bit vào, 256-bit ra) thông qua các tín hiệu điều khiểnstart, init, ready.

Trong khi đó, Bus hệ thống của SoC (Avalon-MM) chỉ giao tiếp với CPU Nios II qua các lệnh Đọc/Ghi (Read/Write) tuần tự theo địa chỉ với độ rộng 32-bit. Công cụ Platform Designer (Qsys) không thể tự động kết nối hai thành phần bất đồng bộ này nếu thiếu mạch logic trung gian.

sha256_avalon_wrapper ra đời như một Avalon Slave Endpoint để giải quyết hai bài toán:

1. **Dịch giao thức (Protocol Bridge):** Chuyển đổi các đường chân thô của lõi thành các tín hiệu bus đúng chuẩn (avs_address, avs_read, avs_write, avs_readdata, avs_writedata) giúp Qsys nhận diện và định tuyến hệ thống.
2. **Bù băng thông (Data Buffer):** Tổ chức một Register File nội bộ để làm nhiệm vụ gom dữ liệu 32-bit tuần tự từ CPU thành khối 512-bit song song cho lõi, đồng thời lưu trữ kết quả 256-bit để CPU đọc ngược lại.

Lớp Wrapper là ngoại vi Avalon Slave chuẩn mực, được quản lý thông qua cơ chế Ánh xạ ô nhớ (Memory-mapped I/O).

RTL của khối wrapper:

Giao diện cổng và Chuẩn bus Avalon-MM (Port Interface)

```
module sha256_avalon_wrapper (
    input  wire        clk,
    input  wire        reset_n,

    // Avalon-MM Slave Interface
    input  wire [4:0]  avs_address,      // 5-bit address for 32 registers
    input  wire        avs_read,
    output reg  [31:0] avs_readdata,
    input  wire        avs_write,
    input  wire [31:0] avs_writedata,
    output wire        avs_waitrequest
);
```

Tín hiệu giao tiếp Avalon-MM Slave

- clk / reset_n: Xung nhịp hệ thống (50 MHz) và reset tích cực mức thấp.
- avs_address[4:0]: Bus địa chỉ 5-bit, định vị chính xác 32 thanh ghi cấu hình nội bộ.
- avs_read / avs_write: Lệnh điều khiển chu trình Đọc/Ghi từ CPU.
- avs_readdata / avs_writedata: Bus dữ liệu 32-bit truyền nhận dữ liệu tuần tự.
- avs_waitrequest = 0: Khóa cố định bằng 0 (Zero-Wait-State), ép bus phản hồi lập tức trong 1 chu kỳ, tối ưu tối đa tốc độ truy xuất của CPU Nios II.

Không gian lưu trữ và Logic Đóng gói dữ liệu (Data Packing)

```
// Internal registers for memory-mapped interface
reg [31:0] data_in_regs [0:15]; // Address 0-15 (512-bit block)
reg start_reg;                  // Address 16 bit 0
reg init_reg;                   // Address 16 bit 1

// Core connections
wire [511:0] block_in;
wire [255:0] hash_out;
wire ready;
wire hash_valid;
// Combine 16x32-bit registers into 512-bit block_in
// SHA256 standard expects Big-Endian format mapping for block_in
genvar i;
generate
   for (i = 0; i < 16; i = i + 1) begin : gen_block_in
      assign block_in[(15-i)*32 +: 32] = data_in_regs[i];
   end
endgenerate
```

Cơ chế ghép khối 512-bit (Big-Endian)

- data_in_regs: Mảng 16 thanh ghi 32-bit, gom dần dữ liệu từ CPU thành khối 512-bit.
- Mạch generate: Gom song song 16 thanh ghi thành một đường bus block_in[511:0] duy nhất.
- Định dạng Big-Endian ((15-i)*32 +: 32): Đảo thứ tự byte khi ghép mạch (Thanh ghi 0 là byte cao nhất, thanh ghi 15 là byte thấp nhất) theo đúng chuẩn FIPS 180-4 của SHA-256.

Liên kết Lõi xử lý (Core Instantiation)

```
// Instantiation of the SHA-256 Core
sha256_core u_sha256_core (
   .clk        (clk),
   .rst_n      (reset_n),
   .start      (start_reg),
   .init       (init_reg),
   .block_in   (block_in),
   .ready      (ready),
   .hash_valid (hash_valid),
   .hash_out   (hash_out)
);
```

Logic Ghi tín hiệu điều khiển và Tạo xung (Write & Pulse Logic)

```
integer j;
   always @(posedge clk or negedge reset_n) begin
      if (!reset_n) begin
         for (j = 0; j < 16; j = j + 1) begin
            data_in_regs[j] <= 32'd0;
         end
         start_reg <= 1'b0;
```

```
        init_reg  <= 1'b0;
    end else begin
    // Auto-clear start signal after 1 cyclce to form a pulse
        if (start_reg) begin
            start_reg <= 1'b0;
        end

        if (avs_write) begin
            if (avs_address <= 5'd15) begin
                // Write to Data Block Registers (0 to 15)
                data_in_regs[avs_address] <= avs_writedata;
            end else if (avs_address == 5'd16) begin
                // Write to Control Register (16)
                start_reg <= avs_writedata[0];
                init_reg  <= avs_writedata[1];
            end
        end
    end
end
```

Khối xử lý logic tuần tự điều khiển quá trình ghi dữ liệu/lệnh từ CPU xuống tập thanh ghi, mang 2 đặc điểm cốt lõi:

- **Phân hoạch địa chỉ ghi (avs_write):**
    - Địa chỉ 0–15: Nạp dữ liệu băm tương ứng vào mảng data_in_regs.
    - Địa chỉ 16: Cấu hình thanh ghi điều khiển (avs_writedata[0] cho start, avs_writedata[1] cho init).
- **Cơ chế tạo xung tự động (Auto-clear pulse):** Khi CPU kích hoạt lệnh start_reg = 1, hệ thống tự động xóa nó về 0 ngay chu kỳ xung nhịp kế tiếp. Cơ chế này tạo ra một xung điều khiển rộng đúng 1 chu kỳ clock, giúp giải phóng CPU khỏi việc phải gửi lệnh xóa thủ công, tối ưu hóa thời gian xử lý của bus hệ thống.

Logic Đọc dữ liệu và Phân rã Bản đồ thanh ghi (Read Logic)

```
// Avalon-MM Read Logic
always @(*) begin
    avs_readdata = 32'd0;
    if (avs_read) begin
        if (avs_address <= 5'd15) begin
            // Read back Data Block Registers
            avs_readdata = data_in_regs[avs_address];
        end else if (avs_address == 5'd16) begin
            // Read Control Register
            avs_readdata = {30'd0, init_reg, start_reg};
        end else if (avs_address == 5'd17) begin
            // Read Status Register
            // Bit 0: ready, Bit 1: hash_valid
            avs_readdata = {30'd0, hash_valid, ready};
        end else if (avs_address >= 5'd18 && avs_address <= 5'd25) begin
            // Read Hash Output Registers (18 to 25)
            // Mapping 256-bit to 8 words (Big-Endian format mapping matching core behavior)
            case(avs_address)
                5'd18: avs_readdata = hash_out[255:224];
```

```
                5'd19: avs_readdata = hash_out[223:192];
                5'd20: avs_readdata = hash_out[191:160];
                5'd21: avs_readdata = hash_out[159:128];
                5'd22: avs_readdata = hash_out[127:96];
                5'd23: avs_readdata = hash_out[95:64];
                5'd24: avs_readdata = hash_out[63:32];
                5'd25: avs_readdata = hash_out[31:0];
            endcase
        end
    end
end

assign avs_waitrequest = 1'b0;
```

Khối mạch tổ hợp (always @(*)) thực hiện ánh xạ không gian địa chỉ Avalon-MM sang tập thanh ghi nội bộ để CPU Nios II đọc dữ liệu, cấu trúc cụ thể gồm:

- **Địa chỉ 0–15 (Data Block):** Đọc ngược lại dữ liệu đầu vào đã nạp trong data_in_regs.
- **Địa chỉ 16 (Control):** Kiểm tra trạng thái lệnh với init_reg (bit 1) và start_reg (bit 0).
- **Địa chỉ 17 (Status - Phục vụ Polling):** Gộp cờ hash_valid (bit 1) và ready (bit 0) vào các bit thấp. CPU Nios II sẽ liên tục kiểm tra (polling) thanh ghi này để nhận biết thời điểm lõi phần cứng hoàn thành tác vụ.
- **Địa chỉ 18–25 (Hash Output):** Sử dụng mạch rẽ nhánh case để chia nhỏ kết quả băm 256-bit (hash_out) từ lõi thành 8 từ mã 32-bit tuần tự. Quá trình cắt bit tuân thủ định dạng Big-Endian (địa chỉ 18 chứa các bit cao nhất [255:224], địa chỉ 25 chứa các bit thấp nhất [31:0]) để phần mềm đọc ra chuỗi băm đúng chuẩn.

Tối ưu bus: Đường tín hiệu avs_waitrequest = 1'b0 ép toàn bộ cấu trúc Register Map trên phản hồi lập tức trong 1 chu kỳ máy (Zero-Wait-State).

Nhận xét:

Lớp bọc là mạch logic trung gian bắt buộc để biến lõi băm thô (sha256_core) thành một **Avalon-MM Slave Endpoint** nhằm giải quyết 3 vấn đề:

- **Ánh xạ bộ nhớ (Memory-Mapped):** Tạo một Register Map gồm 32 địa chỉ giúp phần mềm C (CPU Nios II) điều khiển phần cứng bằng các thao tác ghi/đọc con trỏ (Pointer).
- **Bù độ rộng dữ liệu:** Tự động gom 16 chu kỳ ghi 32-bit từ CPU thành khối 512-bit cấp cho lõi băm. Ngược lại, tách kết quả 256-bit từ lõi thành 8 từ mã 32-bit cho CPU đọc về.
- **Đồng bộ luồng lệnh:** Khi CPU ghi lệnh vào thanh ghi số 16, wrapper tự động tạo một xung kích hoạt (start = 1) rộng đúng 1 chu kỳ clock để khởi động lõi băm theo đúng giản đồ thời gian.

Sha256 core

sha256_core là bộ tăng tốc phần cứng chịu trách nhiệm thực thi thuật toán băm SHA-256 theo chuẩn FIPS 180-4 bằng máy trạng thái (FSM) lặp qua 64 vòng để thay thế mã phần mềm chạy trên CPU.

- **Đầu vào (Inputs):** Xung nhịp hệ thống (clk), dữ liệu khối băm song song **512-bit** (block_in), và các xung điều khiển thô (start, init).
- **Đầu ra (Outputs):** Kết quả chuỗi băm **256-bit** (hash_out) và các cờ trạng thái báo hoàn thành (ready, hash_valid).

CHƯƠNG 3: MÔ PHỎNG, KIỂM THỬ VÀ ĐÁNH GIÁ

I. Kiểm thử SHA-256 – Avalon wrapper IP.

1. Pre-Simulation:

Trong dự án này, quy trình kiểm thử trước khi tổng hợp (pre-simulation) cho IP core SHA256 được thực hiện tự động hóa bằng cách kết hợp giữa SystemVerilog, Python và framework Cocotb.
Mục tiêu của quy trình này là xác minh tính đúng đắn về mặt chức năng của thiết kế (RTL), kiểm tra các trường hợp đặc biệt của giao thức và thu thập dữ liệu về độ bao phủ chức năng (functional coverage) trước khi chuyển sang các bước tổng hợp và triển khai trên FPGA.

1.1 Môi trường kiểm thử (Testbench) với Cocotb

Cocotb (COroutine COsimulation TestBench) là một framework mã nguồn mở dựa trên Python, cho phép viết testbench để kiểm thử các thiết kế HDL (VHDL/Verilog/SystemVerilog) một cách nhanh chóng và linh hoạt.

1.1.1 Thiết lập và Khởi chạy:

Lệnh khởi chạy pre-simulation:

```
python scripts\python\run_sha256_cocotb.py --random-count 100
```

Một script Python (run_sha256_cocotb.py) được sử dụng để tự động hóa toàn bộ quá trình. Script này sẽ thực hiện hai bước chính:
+ Build: Gọi trình mô phỏng (trong đồ án là dùng Icarus Verilog) để biên dịch các file SystemVerilog của IP core SHA256 (sha256Core.sv, sha256Wrapper.sv) cùng với các thư viện cần thiết của Cocotb. Kết quả là một thư viện mô phỏng sẵn sàng để chạy.

+ Test: Khởi chạy trình mô phỏng đã được build, nạp thiết kế và thực thi các test case được viết bằng Python trong file test_sha256_cocotb.py.
Việc sử dụng Cocotb cho phép tận dụng hệ sinh thái mạnh mẽ của Python, bao gồm các thư viện chuẩn như hashlib để tạo ra giá trị tham chiếu (golden model) cho thuật toán SHA-256, và các thư viện khác để xử lý file, tạo dữ liệu ngẫu nhiên và báo cáo kết quả.

1.1.2 Cấu trúc Testbench:

File test_sha256_cocotb.py định nghĩa môi trường kiểm thử. Nó bao gồm các coroutine (hàm async) để tương tác với tín hiệu của thiết kế (DUT - Design Under Test) thông qua giao thức Avalon Memory-Mapped.

Các hàm tiện ích như reset_dut, av_write, av_read được tạo ra để mô phỏng các giao dịch đọc/ghi trên bus Avalon, giúp cho việc viết test case trở nên rõ ràng và dễ dàng hơn.

Một đồng hồ (Clock) được tạo và chạy trong một tiến trình nền để cung cấp xung nhịp cho DUT.

1.2. Các Loại Test Case

Để đảm bảo IP core SHA256 hoạt động chính xác trong mọi tình huống, một bộ test case đa dạng đã được xây dựng, bao gồm cả các ca kiểm thử định hướng (directed tests) và ngẫu nhiên (random tests). Các test case này được định nghĩa và tạo ra trong file sha256_vectors.py.

a. Các Ca Kiểm Thử Định Hướng (Directed Test Cases)

	A	B	C	D	E	F	G	H	I	J
1	suite	case_id	case_name	category	message_len_bytes	num_blocks	expected_digest	actual_digest	pass	error
2	directed_and_random_s	0	empty	directed	0	1	e3b0c44298fc1c14	e3b0c44298fc1c14	PASS	
3	directed_and_random_s	1	one_byte	directed	1	1	ca978112ca1bbdca	ca978112ca1bbdca	PASS	
4	directed_and_random_s	2	abc	directed	3	1	ba7816bf8f01cfea4	ba7816bf8f01cfea4	PASS	
5	directed_and_random_s	3	quick_brown	directed	43	1	d7a8fbb307d78094	d7a8fbb307d78094	PASS	
6	directed_and_random_s	4	all_zero_64	directed	64	2	f5a5fd42d16a20302	f5a5fd42d16a20302	PASS	
7	directed_and_random_s	5	all_ff_64	directed	64	2	8667e718294e9e0d	8667e718294e9e0d	PASS	
8	directed_and_random_s	6	alternating_aa55_64	directed	64	2	d712a59eaf7c09fa9	d712a59eaf7c09fa9	PASS	
9	directed_and_random_s	7	walking_1_64	directed	64	2	26d03219f4a42c8a	26d03219f4a42c8a	PASS	
10	directed_and_random_s	8	walking_0_64	directed	64	2	fd9a545953d5f8bb7	fd9a545953d5f8bb7	PASS	
11	directed_and_random_s	9	len_55	directed	55	1	b0d89fdd8ea17501	b0d89fdd8ea17501	PASS	
12	directed_and_random_s	10	len_56	directed	56	2	db31bd267a4cf128	db31bd267a4cf128	PASS	
13	directed_and_random_s	11	len_57	directed	57	2	7a330d7cd6f2624ef	7a330d7cd6f2624ef	PASS	
14	directed_and_random_s	12	len_63	directed	63	2	29af2686fd53374a3	29af2686fd53374a3	PASS	
15	directed_and_random_s	13	len_64	directed	64	2	fdeab9acf3710362b	fdeab9acf3710362b	PASS	
16	directed_and_random_s	14	len_65	directed	65	2	4bfd2c8b6f1eec7a2	4bfd2c8b6f1eec7a2	PASS	
17	directed_and_random_s	15	len_100	directed	100	2	c87efd8ee1c6706fe	c87efd8ee1c6706fe	PASS	
18	directed_and_random_s	16	len_127	directed	127	3	996e43a7b8b548e3	996e43a7b8b548e3	PASS	
19	directed_and_random_s	17	len_128	directed	128	3	45b96e652b93670a	45b96e652b93670a	PASS	
20	directed_and_random_s	18	len_129	directed	129	3	d3aa8a8ed6e3aee6	d3aa8a8ed6e3aee6	PASS	
21	directed_and_random_s	19	len_1024	directed	1024	17	4633839dbe8e247c	4633839dbe8e247c	PASS	

Hình 3. 1 Các Ca Kiểm Thử Định Hướng (Directed Test Cases)

Đây là những test case được tạo thủ công để nhắm vào các kịch bản cụ thể, các giá trị biên và các trường hợp đặc biệt của thuật toán SHA-256 và giao thức điều khiển.

Kiểm tra các giá trị biên của message:

1. Message rỗng (empty): Kiểm tra khả năng xử lý đầu vào không có dữ liệu.
2. Message ngắn: Các message như "a" (one_byte) và "abc" là các vector kiểm thử tiêu chuẩn của NIST.
3. Message có độ dài quanh điểm thêm padding: Các message có độ dài 55, 56, 57, 63, 64, 65 bytes. Các độ dài này được chọn để kiểm tra logic padding của SHA-256. *Ví dụ, message 55 byte cần padding và nằm trong một block, trong khi message 56 byte sẽ khiến phần padding và độ dài phải tràn sang một block thứ hai.*

4. Message nhiều block: Các message dài hơn 64 byte (len_100, len_1024,...) để đảm bảo IP core có thể xử lý chuỗi nhiều block dữ liệu một cách chính xác, cập nhật giá trị hash trung gian sau mỗi block.

Kiểm tra các mẫu dữ liệu đặc biệt:

2. Toàn bit 0 (all_zero_64) hoặc toàn bit 1 (all_ff_64): Giúp phát hiện các lỗi liên quan đến các giá trị hằng số hoặc các phép toán logic.
3. Mẫu xen kẽ (alternating_aa55_64): Kiểm tra khả năng xử lý các mẫu dữ liệu có tính chu kỳ.
4. Mẫu walking-1 và walking-0: Dữ liệu có một bit 1 (hoặc 0) di chuyển qua các vị trí trong một byte/word. Loại test này rất hiệu quả trong việc phát hiện lỗi trên các đường bus dữ liệu hoặc trong các thanh ghi.

b. Các Ca Kiểm Thử Ngẫu Nhiên (Random Test Cases)

22	directed_and_random_s	20	random_000_len_111	random	1113	18	251adecdaef6e815(	251adecdaef6e815c PASS
23	directed_and_random_s	21	random_001_len_187	random	1874	30	59553fae3fb3d976e	59553fae3fb3d976e PASS
24	directed_and_random_s	22	random_002_len_163	random	1633	26	9200abb590a9fcf50	9200abb590a9fcf50 PASS
25	directed_and_random_s	23	random_003_len_123	random	1234	20	f831f51e67e6c96fc:	f831f51e67e6c96fc: PASS
26	directed_and_random_s	24	random_004_len_183	random	1838	29	d7a854a1d110cc73	d7a854a1d110cc73(PASS
27	directed_and_random_s	25	random_005_len_60	random	60	2	b4868b8192ec5ee6	b4868b8192ec5ee6: PASS
28	directed_and_random_s	26	random_006_len_576	random	576	10	9eddedb1c05aa3f9!	9eddedb1c05aa3f9! PASS
29	directed_and_random_s	27	random_007_len_125	random	1257	20	9d88ad56d4a71b40	9d88ad56d4a71b40: PASS
30	directed_and_random_s	28	random_008_len_107	random	1077	17	fcb1fd931a34c753e	fcb1fd931a34c753e PASS
31	directed_and_random_s	29	random_009_len_118	random	1180	19	ac1c1479dfada2f71	ac1c1479dfada2f71 PASS
32	directed_and_random_s	30	random_010_len_765	random	765	13	e13f0479cecf2e63c	e13f0479cecf2e63c PASS
33	directed_and_random_s	31	random_011_len_192	random	1922	31	9e1d5757c1f46755!	9e1d5757c1f467555 PASS
34	directed_and_random_s	32	random_012_len_140	random	1408	23	4974a9e894ac0db0	4974a9e894ac0db0(PASS
35	directed_and_random_s	33	random_013_len_119	random	1196	19	edffab2749df3f4832	edffab2749df3f4832 PASS
36	directed_and_random_s	34	random_014_len_157	random	1577	25	eff24c0b8d0fbf6a2{	eff24c0b8d0fbf6a28 PASS
37	directed_and_random_s	35	random_015_len_149	random	1496	24	9774baeb09fb7441(	9774baeb09fb7441c PASS
38	directed_and_random_s	36	random_016_len_110	random	1108	18	939bb5e04c17dff06	939bb5e04c17dff06 PASS
39	directed_and_random_s	37	random_017_len_353	random	353	6	be24cfeb31843265{	be24cfeb318432658 PASS
40	directed_and_random_s	38	random_018_len_118	random	1189	19	3c01c520b1e9aa11	3c01c520b1e9aa11(PASS
41	directed_and_random_s	39	random_019_len_156	random	1562	25	e3ddb7a628439ffd6	e3ddb7a628439ffd6 PASS
42	directed_and_random_s	40	random_020_len_204	random	2044	33	8ec28381d4021f51f	8ec28381d4021f51f PASS
43	directed_and_random_s	41	random_021_len_143	random	1430	23	f2fdb8ee4e46a3a01	f2fdb8ee4e46a3a01 PASS
44	directed_and_random_s	42	random_022_len_451	random	451	8	b51a28fa2f21f445d!	b51a28fa2f21f445d! PASS
45	directed_and_random_s	43	random_023_len_158	random	158	3	5c386496934785f5(	5c386496934785f5e PASS
46	directed_and_random_s	44	random_024_len_608	random	608	10	7bedc49b98efec3dl	7bedc49b98efec3dl PASS
47	directed_and_random_s	45	random_025_len_28	random	28	1	b3ad7d6b7e046891	b3ad7d6b7e046891(PASS
48	directed_and_random_s	46	random_026_len_138	random	1381	22	b28f346a7ebf09d3b	b28f346a7ebf09d3b PASS
49	directed_and_random_s	47	random_027_len_161	random	1615	26	9e2353f69e2abc63(	9e2353f69e2abc63(PASS

Hình 3. 2 Các Ca Kiểm Thử Ngẫu Nhiên (Random Test Cases)

Để bao phủ không gian trạng thái rộng lớn mà các test case định hướng có thể bỏ sót, phương pháp kiểm thử ngẫu nhiên được áp dụng.
Tạo message ngẫu nhiên: Script sẽ tạo ra một số lượng lớn (ví dụ: 64) các message với độ dài ngẫu nhiên (từ 0 đến 2048 byte) và nội dung ngẫu nhiên.

Ưu điểm: Phương pháp này giúp tìm ra các lỗi không lường trước (corner-case bugs) mà người thiết kế testbench có thể không nghĩ tới. Việc sử dụng một seed (hạt giống) cho bộ sinh số ngẫu nhiên đảm bảo rằng các bộ test case ngẫu nhiên này có thể được tái tạo lại để gỡ lỗi nếu phát hiện ra sự cố.

c. Các Ca Kiểm Thử Giao Thức (Protocol Test Cases)

Ngoài việc kiểm tra logic thuật toán, testbench còn tập trung vào việc xác minh hoạt động của giao thức điều khiển IP core qua bus Avalon.

1022	directed_byteenable_an	0	byteenable_low_byte	protocol					PASS
1023	directed_byteenable_an	1	irq_pending_clear_aft	protocol	3	1	ba7816bf8f01cfea4	ba7816bf8f01cfea4	PASS
1024	protocol_corner_cases	0	invalid_address_read	protocol					PASS
1025	protocol_corner_cases	1	read_before_done_an	protocol	3	1	ba7816bf8f01cfea4	ba7816bf8f01cfea4	PASS
1026	protocol_corner_cases	2	reset_while_busy	protocol					PASS
1027	protocol_corner_cases	3	back_to_back_1	protocol	18	1	8507ec81f6070a34	8507ec81f6070a34	PASS
1028	protocol_corner_cases	4	back_to_back_2	protocol	19	1	69b4e82c065d4382	69b4e82c065d4382	PASS

Hình 3. 3 Các Ca Kiểm Thử Giao Thức (Protocol Test Cases)

1. Kiểm tra byteenable: Đảm bảo việc ghi từng byte vào các thanh ghi 32-bit hoạt động chính xác.
2. Xóa ngắt (IRQ clear): Xác minh rằng cờ ngắt được xóa đúng cách sau khi CPU đọc kết quả.
3. Đọc/ghi địa chỉ không hợp lệ: Kiểm tra IP core trả về giá trị 0 khi đọc một địa chỉ không được định nghĩa, tuân thủ đặc tả bus.
4. Bắt đầu khi đang bận (start_while_busy): Thử gửi lệnh bắt đầu mới trong khi IP core đang xử lý một message. Test case này xác minh rằng IP core sẽ bỏ qua lệnh mới và không bị rơi vào trạng thái lỗi.
5. Reset khi đang bận (reset_while_busy): Kiểm tra khả năng reset bất đồng bộ của IP core, đảm bảo nó trở về trạng thái sẵn sàng (idle) một cách an toàn từ giữa một quá trình tính toán.
6. Giao dịch liên tiếp (back-to-back): Kiểm tra khả năng xử lý một message mới ngay sau khi hoàn thành message trước đó, nhằm đánh giá hiệu suất và tính ổn định.

1.3. So Sánh Kết Quả và Báo Cáo

Đối với mỗi message đầu vào, testbench sẽ tính toán giá trị hash golden digest bằng hàm *hashlib.sha256()* của Python. Sau khi IP core hoàn tất, giá trị hash do nó tạo ra sẽ được đọc về và so sánh với giá trị vàng. Bất kỳ sự khác biệt nào cũng sẽ được báo cáo là lỗi.

Báo cáo CSV: Mọi kết quả kiểm thử (thành công hay thất bại) đều được ghi lại vào một file CSV *(sha256_pre_sim_results.csv)*. Mỗi hàng trong file CSV chứa thông tin chi tiết về một test case: tên, loại (directed/random), độ dài message, số block, giá trị hash kỳ vọng, giá trị hash thực tế, trạng thái PASS/FAIL và thông báo lỗi nếu có. File báo cáo này rất hữu ích cho việc theo dõi tiến trình và phân tích lỗi.

2. Coverage:

Module ID	Module Name	Instance[s] Count	Hierarchical Instance[s]	Statement Score	Branch Score	Condition Score	Toggle Score	Module definition in File	File ID
1	tb_sha256_vectors	1	tb_sha256_vectors	77.095	51.2195	10	72.55	C:/Users/dinhkhanhdang/Do_An_Thay_NGUYEN/testbench/tb_sha256 ..	1
2	sha256_avalon_wrapper	1	tb_sha256_vectors.dut	100	96.7742	75	58.36	C:/Users/dinhkhanhdang/Do_An_Thay_NGUYEN/sha256Wrapper.sv	2
3	sha256_core	1	tb_sha256_vectors.dut.u_sha256_core	100	90	100	49.91	C:/Users/dinhkhanhdang/Do_An_Thay_NGUYEN/sha256Core.sv	3
4	glbl	1	glbl	100	0	0	6.57	C:/Users/dinhkhanhdang/Do_An_Thay_NGUYEN/vivado/SHA_RISCV_KL ..	4

Hình 3. 4 Coverage

2.1 Đối với SHA_Core:

2.1.1. Điểm số Coverage:

- **Statement Coverage**: 100%
- **Branch Coverage**: 90%
- **Condition Coverage**: 100%
- **Toggle Coverage**: 49.91%

2.1.2. Nhận định về Code và Testbench:

- **Statement và Condition Coverage (100%) & Branch Coverage (90%)**: Mọi dòng lệnh và điều kiện logic trong module đều đã được kiểm thử, cho thấy testbench đã bao phủ tốt các luồng tính toán cơ bản của thuật toán.
- **Toggle Coverage (49.91%)**: Tín hiệu như ***start, init,*** các thanh ghi ***a, b, c, e, f, g, h,*** và các dây trung gian t1, t2 không được toggle đầy đủ. Điều này cho thấy dữ liệu đầu vào và các kịch bản điều khiển còn đơn điệu, chưa đủ đa dạng để tạo ra các giá trị khác nhau trong quá trình tính toán, dẫn đến nguy cơ bỏ sót các lỗi liên quan đến phần cứng (hardware-specific bugs) như race condition hay glitch.

2.1.3. Hướng khắc phục và cải thiện:

Để đạt được coverage toàn diện hơn nhóm có thể cần cải thiện testbench theo các hướng sau:

Cải thiện Branch Coverage: Thêm vào testbench các kịch bản mà trong đó module sha256_core ở trạng thái IDLE trong nhiều chu kỳ clock mà không có tín hiệu start được kích hoạt. Điều này đảm bảo lõi hoạt động đúng khi ở trạng thái chờ.

Tăng Toggle Coverage:

+ Đa dạng hóa dữ liệu đầu vào (block_in) bằng các bộ test ngẫu nhiên (constrained-random tests) thay vì chỉ dùng các vector cố định. Sử dụng các mẫu dữ liệu đặc biệt như toàn 0, toàn 1, walking-ones, walking-zeros.

+ Tạo ra các kịch bản điều khiển ngẫu nhiên, ví dụ như trì hoãn (delay) ngẫu nhiên giữa các lần băm để các thanh ghi nội bộ có cơ hội được reset hoặc thay đổi giá trị.

+ Thêm các bài test reset ngẫu nhiên (reset injection) khi lõi đang tính toán để kiểm tra khả năng phục hồi của thiết kế.

2.2 Đối với Wrapper - SHA

2.2.1. Điểm số Coverage:

- **Statement Coverage**: 100%
- **Branch Coverage**: 96.77%
- **Condition Coverage**: 75%
- **Toggle Coverage**: 58.36%

2.2.2. Nhận định về Code và Testbench:

- **Statement và Branch Coverage (100% và 96.77%)**: Hầu hết các logic điều khiển, ghi/đọc thanh ghi đều đã được thực thi.
- **Condition Coverage (75%)**: Báo cáo cho thấy điều kiện *(avs_address == 5'b10001)__FALSE (dòng 90)* và *(avs_address >= 5'b10010)__FALSE* chưa được kiểm tra. Điều này có nghĩa là testbench chưa bao giờ thực hiện ghi vào một địa chỉ không phải là thanh ghi điều khiển (0x10) hoặc thanh ghi trạng thái (0x11), và chưa bao giờ đọc một địa chỉ không phải là vùng digest (0x12 - 0x19).
- **Toggle Coverage (58.36%)**: Tương tự như sha256_core, điểm số này còn thấp. Các tín hiệu đầu vào từ bus như avs_byteenable, avs_writedata và các tín hiệu nội bộ như write_en, hash_valid_d chưa được toggle đầy đủ. Điều này cho thấy các giao dịch trên bus chưa đủ đa dạng.

2.2.3. Hướng khắc phục và cải thiện:

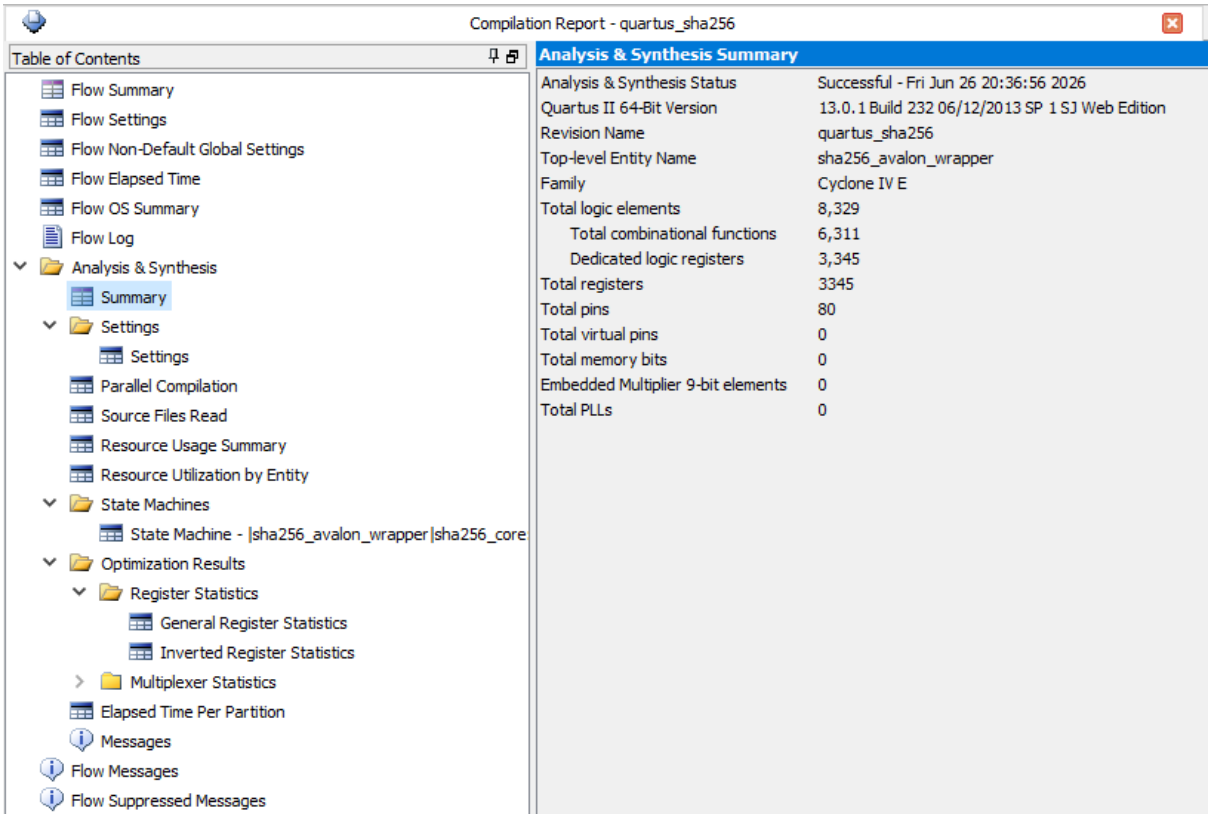
- **Hoàn thiện Branch và Condition Coverage**:

+ Thêm các bài test thực hiện ghi/đọc vào các địa chỉ không hợp lệ (ví dụ: 0x1C, 0x1F) để đảm bảo module xử lý đúng (thường là bỏ qua hoặc trả về 0). Điều này sẽ bao phủ các nhánh else và default còn thiếu.
+ Thực hiện các giao dịch ghi có avs_byteenable khác 4'b1111 và 4'b0000 để kiểm tra logic ghi từng byte một cách triệt để hơn.

- **Tăng Toggle Coverage**:

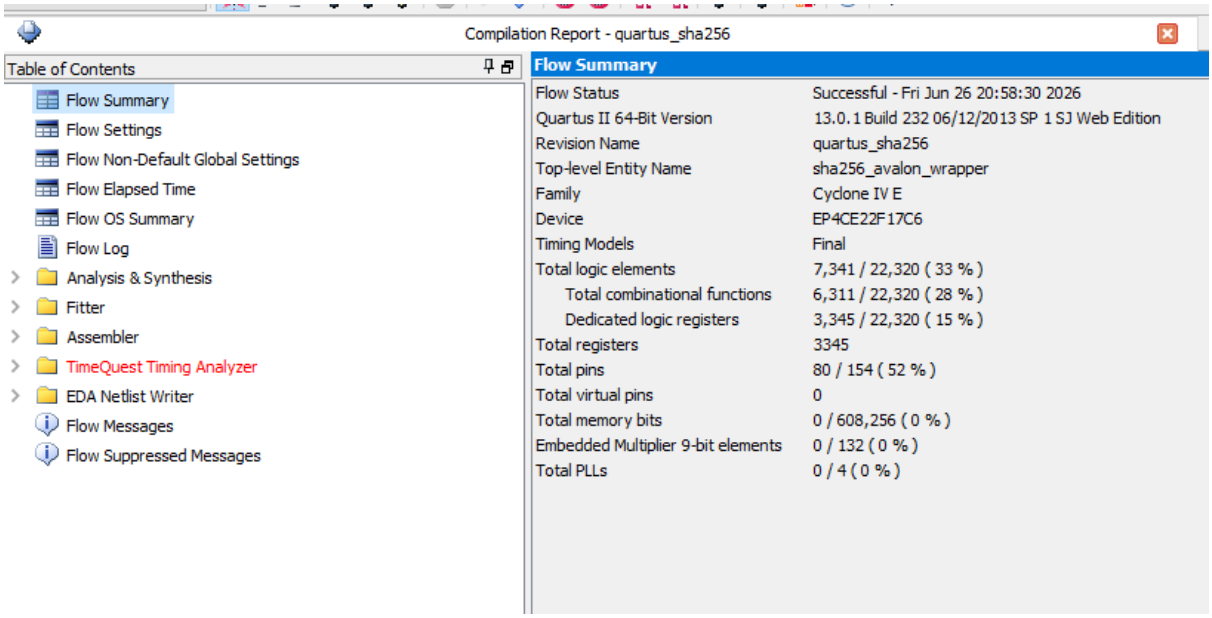
+ Tăng số lượng các kịch bản ghi/đọc ngẫu nhiên lên các thanh ghi với dữ liệu và thứ tự ngẫu nhiên.

3. Synthesis:



Hình 3. 5 Synthesis

Sau bước Analysis & Synthesis, Quartus ước lượng thiết kế sử dụng 8.329 logic elements, trong đó có 6.311 hàm tổ hợp và 3.345 thanh ghi logic chuyên dụng. Thiết kế sử dụng tổng cộng 80 chân I/O, không sử dụng virtual pin, không dùng bộ nhớ nhúng, không dùng phần tử nhân DSP 9-bit và không dùng PLL. Điều này cho thấy kiến trúc SHA-256 hiện tại được ánh xạ chủ yếu bằng logic tổ hợp và thanh ghi, không phụ thuộc vào RAM block, DSP hay tài nguyên clock đặc biệt.

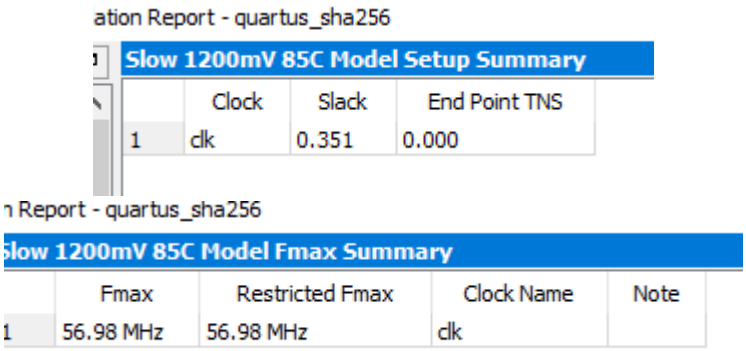


Hình 3. 6 Ước lượng tài nguyên

Sau Fitter, thiết kế sử dụng 7.341 / 22.320 logic elements, tương đương 33% tài nguyên logic của EP4CE22F17C6. Trong đó có 3.996 logic element chỉ chứa logic tổ hợp, 1.030 logic element chỉ chứa thanh ghi, và 2.315 logic element chứa cả LUT và register. Số thanh ghi sử dụng là 3.345 / 23.018, tương đương 15%. Thiết kế dùng 80 / 154 chân I/O, tương đương 52%. Các tài nguyên bộ nhớ M9K, block memory, DSP multiplier và PLL đều bằng 0%, xác nhận thiết kế không tiêu tốn tài nguyên bộ nhớ nhúng hay khối tính toán chuyên dụng.

Xét theo phân cấp thiết kế, toàn bộ sha256_avalon_wrapper sử dụng 7.341 logic cells và 3.345 thanh ghi. Riêng lõi sha256_core:u_sha256_core chiếm phần lớn tài nguyên với 6.270 logic cells và 2.828 thanh ghi. Phần wrapper Avalon bên ngoài sử dụng phần tài nguyên còn lại, chủ yếu phục vụ giao tiếp bus, thanh ghi dữ liệu, điều khiển đọc/ghi và tín hiệu trạng thái. Như vậy, phần lõi SHA-256 là thành phần chiếm tài nguyên chính của toàn hệ thống.

4. Implementation:



	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	0.351	sha256_core:u_sha256_core|round[2]	sha256_core:u_sha256_core|a[31]	clk	clk	17.900	-0.048	17.496
2	0.655	sha256_core:u_sha256_core|round[1]	sha256_core:u_sha256_core|a[31]	clk	clk	17.900	-0.049	17.191
3	0.676	sha256_core:u_sha256_core|round[2]	sha256_core:u_sha256_core|a[29]	clk	clk	17.900	-0.050	17.169
4	0.792	sha256_core:u_sha256_core|round[3]	sha256_core:u_sha256_core|a[31]	clk	clk	17.900	-0.049	17.054
5	0.798	sha256_core:u_sha256_core|w_mem[23][12]	sha256_core:u_sha256_core|a[31]	clk	clk	17.900	-0.063	17.034
6	0.828	sha256_core:u_sha256_core|round[0]	sha256_core:u_sha256_core|a[31]	clk	clk	17.900	-0.049	17.018
7	0.829	sha256_core:u_sha256_core|w_mem[8][15]	sha256_core:u_sha256_core|a[31]	clk	clk	17.900	-0.055	17.011
8	0.980	sha256_core:u_sha256_core|round[1]	sha256_core:u_sha256_core|a[29]	clk	clk	17.900	-0.051	16.864
9	1.024	sha256_core:u_sha256_core|round[2]	sha256_core:u_sha256_core|a[24]	clk	clk	17.900	-0.050	16.821
10	1.060	sha256_core:u_sha256_core|w_mem[47][26]	sha256_core:u_sha256_core|a[31]	clk	clk	17.900	-0.055	16.780
11	1.092	sha256_core:u_sha256_core|round[2]	sha256_core:u_sha256_core|a[28]	clk	clk	17.900	-0.048	16.755

Hình 3. 7 Fmax và Critical Path

Kết quả TimeQuest cho thấy thiết kế đạt timing với tất cả các slack đều dương và TNS bằng 0. Ở mô hình chậm 1200 mV, 85°C, setup slack xấu nhất là 0,351 ns, hold slack xấu nhất là 0,357 ns, và minimum pulse width slack là 8,605 ns. Fmax báo cáo cho clock clk là 56,98 MHz, cao hơn tần số yêu cầu 55,87 MHz, nghĩa là thiết kế đạt yêu cầu hoạt động ở chu kỳ 17,9 ns. Đường setup tới hạn nằm trong lõi SHA-256, từ thanh ghi round[2] đến thanh ghi a[31], với data delay khoảng 17,496 ns.

5. Post – Simulation:

5.1 Run post synthesis simulation functional

```
vlog -sv ../../../testbench/tb_sha256_vectors.sv
vsim work.tb_sha256_vectors +DIRECTED_ONLY
add wave -r /tb_sha256_vectors/*
run -all
```

```
ModelSim> vsim work.tb_sha256_vectors +DIRECTED_ONLY
# vsim work.tb_sha256_vectors "+DIRECTED_ONLY"
# Start time: 21:33:42 on Jun 26,2026
# Loading sv_std.std
# Loading work.tb_sha256_vectors
# Loading work.sha256_avalon_wrapper
# Loading work.sha256_core
# ** Warning: (vsim-8311) System Verilog assertions are supported only in Questasim.
VSIM 4> add wave -r /tb_sha256_vectors/*
VSIM 5> run -all
# [INFO] Running directed Avalon wrapper coverage tests
# [SUMMARY] directed_only=1 failures=0
# ** Note: $finish    : ../../../testbench/tb_sha256_vectors.sv(410)
#    Time: 1420 ns  Iteration: 1  Instance: /tb_sha256_vectors
# 1
# Break in Module tb_sha256_vectors at ../../../testbench/tb_sha256_vectors.sv line 410

VSIM 6>
```

Hình 3. 8 Post functional synthesis

[SUMMARY] directed_only=1 failures=0
Nghĩa là testbench đã chạy xong, các directed test cho Avalon wrapper đều pass, không có failure.

5.2 Run post implementation simulation timing

```
quit -sim

vmap altera_ver C:/intelFPGA/20.1/modelsim_ase/altera/verilog/altera
vmap lpm_ver C:/intelFPGA/20.1/modelsim_ase/altera/verilog/220model
```

```
vmap sgate_ver C:/intelFPGA/20.1/modelsim_ase/altera/verilog/sgate
vmap cycloneive_ver C:/intelFPGA/20.1/modelsim_ase/altera/verilog/cycloneive

vdel -lib work -all
vlib work

vlog quartus_sha256_6_1200mv_85c_slow.svo
vlog -sv +define+GATE_LEVEL_SIM ../../../testbench/tb_sha256_vectors.sv

vsim -L altera_ver -L lpm_ver -L sgate_ver -L cycloneive_ver -sdftyp
/tb_sha256_vectors/dut=quartus_sha256_6_1200mv_85c_v_slow.sdo
work.tb_sha256_vectors +DIRECTED_ONLY +SLOW_CLOCK

add wave -r /tb_sha256_vectors/*
run -all
```

```
ModelSim> vsim -L altera_ver -L lpm_ver -L sgate_ver -L cycloneive_ver -sdftyp /tb_sha256_vectors/dut=quartus_sha256_6_1200mv_85c_v_slow.sdo work.tb_sha256_vectors +DIRECTED_ONLY +SLOW_CLOCK
# vsim -L altera_ver -L lpm_ver -L sgate_ver -L cycloneive_ver -sdftyp "/tb_sha256_vectors/dut=quartus_sha256_6_1200mv_85c_v_slow.sdo" work.tb_sha256_vectors "+DIRECTED_ONLY" "+SLOW_CLOCK"
# Start time: 21:41:20 on Jun 26,2026
# Loading sv_std.std
# Loading work.tb_sha256_vectors
# Loading work.sha256_avalon_wrapper
# Loading cycloneive_ver.cycloneive_lcell_comb
# Loading altera_ver.dffeas
# Loading cycloneive_ver.cycloneive_io_ibuf
# Loading cycloneive_ver.cycloneive_clkctrl
# Loading cycloneive_ver.cycloneive_mux41
# Loading cycloneive_ver.cycloneive_ena_reg
# Loading cycloneive_ver.cycloneive_io_obuf
# Loading instances from quartus_sha256_6_1200mv_85c_v_slow.sdo
# Loading instances from quartus_sha256_6_1200mv_85c_v_slow.sdo
# Loading altera_ver.PRIM_GDFF_LOW
# ** Warning: Design size of 24995 statements exceeds ModelSim-Intel FPGA Starter Edition recommended capacity.
# Expect performance to be adversely affected.
# Loading timing data from quartus_sha256_6_1200mv_85c_v_slow.sdo
# Loading timing data from quartus_sha256_6_1200mv_85c_v_slow.sdo
# ** Note: (vsim-3587) SDF Backannotation Successfully Completed.
#    Time: 0 ps  Iteration: 0  Instance: /tb_sha256_vectors File: ../../../testbench/tb_sha256_vectors.sv

VSIM 31> add wave -r /tb_sha256_vectors/*
VSIM 32> run -all
# [INFO] Running directed Avalon wrapper coverage tests
# [SUMMARY] directed_only=1 failures=0
# ** Note: $finish    : ../../../testbench/tb_sha256_vectors.sv(410)
#    Time: 14200 ns  Iteration: 1  Instance: /tb_sha256_vectors
# 1
# Break in Module tb_sha256_vectors at ../../../testbench/tb_sha256_vectors.sv line 410

VSIM 33>
```

Hình 3. 9 Post Implementation timing

II. Kiểm thử RISC-V

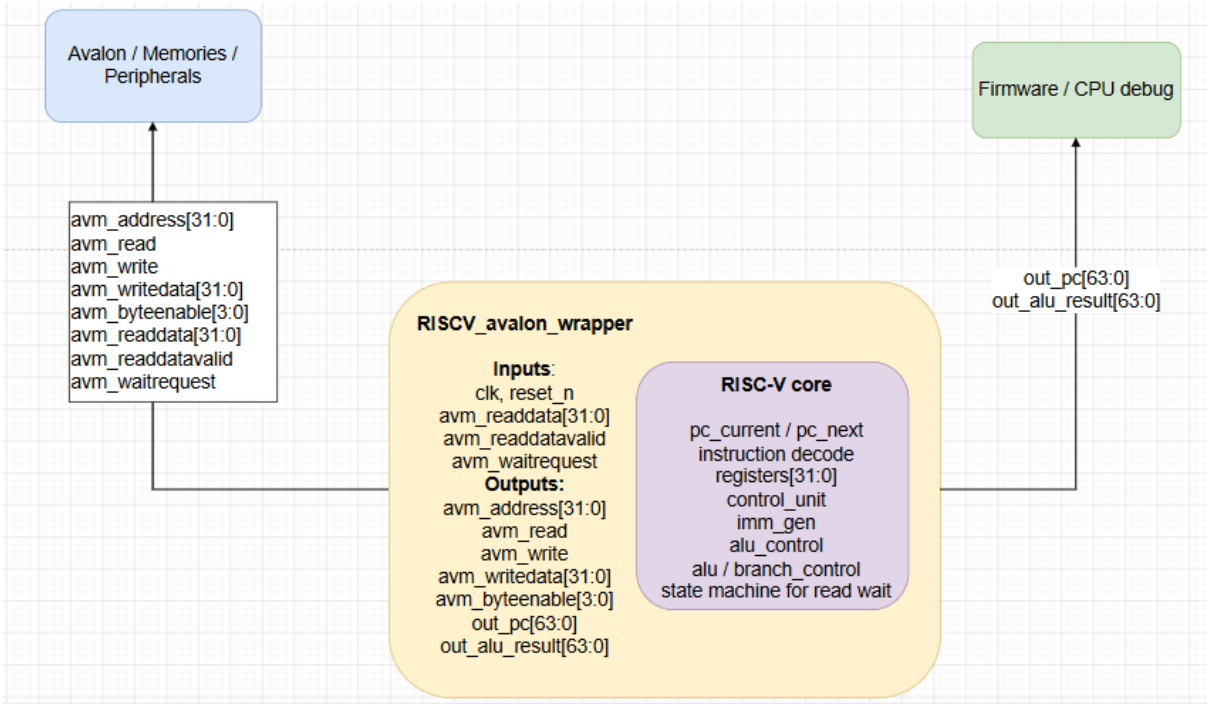
Pre-Simulation.

Khối RISC-V của nhóm là khối single-cycle tự custom, phần RISC-V base hỗ trợ khoảng 31 lệnh kiến trúc:

Loại lệnh	**Các lệnh**	**Số lượng**
Arithmetic/Logical (Reg-Reg)	ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT, SLTU	10
Arithmetic/Logical (Imm)	ADDI, ANDI, ORI, XORI, SLTI, SLTIU, SLLI, SRLI, SRAI	9
Branch	BEQ, BNE, BLT, BGE, BLTU, BGEU	6

Loại lệnh	Các lệnh	Số lượng
Jump/Upper Imm	JAL, JALR, LUI, AUIPC	4
Memory	Load, Store	2
Tổng		**31**

Trong giới hạn đồ án này, khối RISCV được xây dựng như một bộ xử lý RISC-V 64-bit đơn giản, phục vụ vai trò điều khiển hệ thống và chạy firmware kiểm thử thuật toán SHA-256. Thiết kế không hướng đến việc hỗ trợ đầy đủ toàn bộ chuẩn RV64I, mà tập trung vào một tập lệnh con đủ để thực hiện các phép toán số học, logic, rẽ nhánh, truy cập bộ nhớ và giao tiếp với các ngoại vi trong hệ thống. Kiểm thử RISC-V chạy firmware.



Hình 3. 10 Cấu trúc RISCV

Trong flow hệ thống thực tế, file quan trọng là **riscv_avalon_wrapper.sv**, vì đây là phiên bản CPU được bọc giao tiếp Avalon Master để có thể truy cập các ngoại vi trong hệ thống Qsys/Platform Designer.

riscv_avalon_wrapper đóng vai trò cầu nối giữa CPU RISC-V và hệ thống bus Avalon. Wrapper xuất các tín hiệu Avalon Master gồm địa chỉ, read, write, writedata, byteenable, waitrequest và readdatavalid. Khi firmware thực hiện lệnh load hoặc store, địa chỉ được lấy từ kết quả ALU, dữ liệu ghi lấy từ thanh ghi nguồn rs2, còn dữ liệu đọc từ Avalon được ghi ngược về thanh ghi đích rd. Do bus Avalon trong thiết kế đang rộng 32 bit, dữ liệu đọc được zero-extend lên 64 bit trước khi ghi vào register file, còn dữ liệu ghi ra

bus lấy 32 bit thấp của thanh ghi nguồn. Vì vậy firmware hiện tại sử dụng chủ yếu các thao tác kiểu word như lw và sw.

Pre-simulation:

```
# [ROUND_00] cycle=16 pc=0000000000000040 instr=0064da93
# [ROUND_00] work a..h = 6a09e667 bb67ae85 3c6ef372 a54ff53a 510e527f 9b05688c 1f83d9ab 5be0cd19
# [ROUND_00] digest a0-a7 = 00000000000000000000000000000000000000000000000000000000000000000000000000000000
# [ROUND_08] cycle=769 pc=0000000000000c04 instr=0064da93
# [ROUND_08] work a..h = 85a07b5f e5030380 2b4209f5 04409a6a 0c657a79 9b27a401 714260ad 43ada245
# [ROUND_08] digest a0-a7 = 00000000000000000000000000000000000000000000000000000000000000000000000000000000
# [ROUND_16] cycle=1521 pc=00000000000017c4 instr=0064da93
# [ROUND_16] work a..h = b0fa238e c0645fde d932eb16 87912990 07590dcd 0b92f20c 745a48de 1e578218
# [ROUND_16] digest a0-a7 = 00000000000000000000000000000000000000000000000000000000000000000000000000000000
# [ROUND_24] cycle=2280 pc=00000000000023a0 instr=0064da93
# [ROUND_24] work a..h = c5d53d8d a7a3623f c2606d6d 9dc68b63 aa47c347 49f5114a e1257970 8ada8930
# [ROUND_24] digest a0-a7 = 00000000000000000000000000000000000000000000000000000000000000000000000000000000
# [ROUND_32] cycle=3040 pc=0000000000002f80 instr=0064da93
# [ROUND_32] work a..h = 73b33bf5 ea992a22 a0060b30 363482c9 ba591112 0109ab3a ade79437 6112a3b7
# [ROUND_32] digest a0-a7 = 00000000000000000000000000000000000000000000000000000000000000000000000000000000
# [ROUND_40] cycle=3800 pc=0000000000003b60 instr=0064da93
# [ROUND_40] work a..h = 79ea687a 6dc57a8a 34df1604 41a65cb1 1efbc0a0 f0781bc8 a507a53d 0772a26b
# [ROUND_40] digest a0-a7 = 00000000000000000000000000000000000000000000000000000000000000000000000000000000
# [ROUND_48] cycle=4560 pc=0000000000004740 instr=0064da93
# [ROUND_48] work a..h = 4172328d a14c14b0 72ab4b91 26628815 fecf0bc6 d57b94a9 b7755da1 a80f11f0
# [ROUND_48] digest a0-a7 = 00000000000000000000000000000000000000000000000000000000000000000000000000000000
# [ROUND_56] cycle=5320 pc=0000000000005320 instr=0064da93
# [ROUND_56] work a..h = 38cc9913 3ec45cdb f5702fdb 101fd28f 54cb266b e50e1b4f 9f4787c3 529e7d00
# [ROUND_56] digest a0-a7 = 00000000000000000000000000000000000000000000000000000000000000000000000000000000
# [ROUND_63] cycle=5985 pc=0000000000005d84 instr=0064da93
# [ROUND_63] work a..h = d39a2165 04d24d6c b85e2ce9 b6ae8fff fb121210 948d25b6 961f4894 b21bad3d
# [ROUND_63] digest a0-a7 = 00000000000000000000000000000000000000000000000000000000000000000000000000000000
# [DIGEST_START] cycle=6080 pc=0000000000005f00 instr=6a09ee37
# [DIGEST_START] work a..h = 506e3058 d39a2165 04d24d6c b85e2ce9 5ef50f24 fb121210 948d25b6 961f4894
# [DIGEST_START] digest a0-a7 = 00000000000000000000000000000000000000000000000000000000000000000000000000000000
# [TB] final cycle=6118 pc=0000000000005f98 instr=02089893 alu=f20015ad00000000
# [TB] final a0-a7/x10-x17 = ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
# [TB] expected             = ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
# [TB] x31/t6 marker        = 00000000
# [PASS] SHA256(abc) digest matched in a0-a7/x10-x17.
# [DIGEST_READY] cycle=6120 pc=0000000000005fa0 instr=600e0fb7
# [DIGEST_READY] work a..h = 506e3058 d39a2165 04d24d6c b85e2ce9 5ef50f24 fb121210 948d25b6 961f4894
# [DIGEST_READY] digest a0-a7 = ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
# [MARKER] cycle=6122 pc=0000000000005fa8 x31/t6=600dfeed
# [DONE_LOOP] cycle=6122 pc=0000000000005fa8 digest_match=1 marker=600dfeed
```

Hình 3. 11 Presimulation RISC-V

Output này cho thấy ModelSim đã load các module RTL của RISC-V core, sau đó testbench nạp firmware_abc.hex vào instruction memory và chạy mô phỏng CPU-only để tính SHA-256 cho input hardcode "abc".

Ở cycle 16, firmware đã chuẩn bị đúng block SHA-256 đầu vào với 61626380 và độ dài 0x18 bit; các mốc [ROUND_00] đến [ROUND_63] là snapshot của 8 biến làm việc a..h trong thuật toán SHA-256, cho thấy CPU đang thực thi thật 64 round của thuật toán, trong khi a0-a7 vẫn bằng 0 vì digest cuối chưa được ghi. Đến [DIGEST_START], firmware bắt đầu cộng trạng thái cuối với initial hash để tạo digest; tại cycle 6118 testbench thấy a0-a7/x10-x17 đã bằng đúng giá trị chuẩn ba7816bf...f20015ad, nên báo [PASS]. Sau đó ở [DIGEST_READY], digest được xác nhận nằm đầy đủ trong a0-a7, rồi firmware ghi marker t6/x31 = 600dfeed và nhảy vào vòng lặp done, chứng minh chương trình đã hoàn tất đúng và $finish chỉ là testbench kết thúc mô phỏng bình thường.

Synthesis & Implementation



Hình 3. 12 Fmax RISC-V

2.1. Cải tiến lõi RISC-V theo kiến trúc RV32I 5-stage pipeline

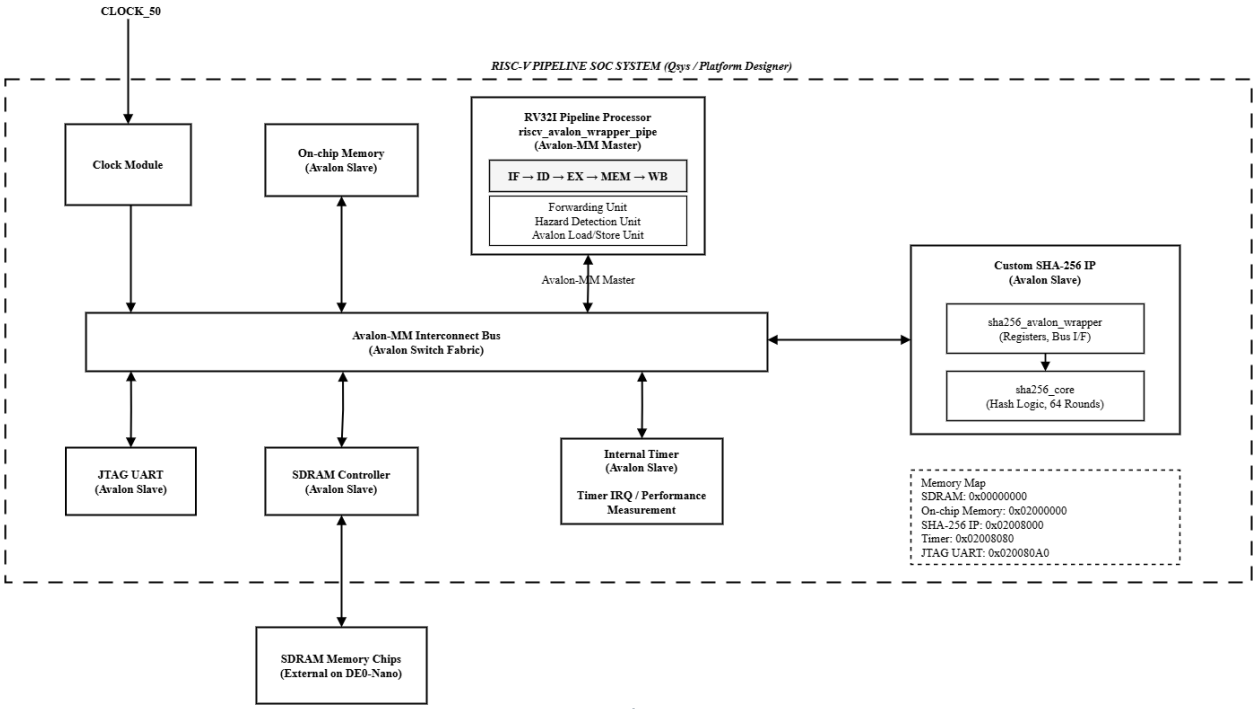
2.1.1. Mục tiêu cải tiến

Trong kiến trúc ban đầu của đồ án, lõi RISC-V đóng vai trò là bộ xử lý điều khiển trung tâm, thực thi firmware và truy cập các ngoại vi như SDRAM, JTAG UART, Timer và khối tăng tốc SHA-256 thông qua giao tiếp Avalon-MM. Cách tổ chức này phù hợp với mô hình hardware/software co-design: CPU đảm nhiệm điều phối luồng dữ liệu, còn khối SHA-256 chuyên dụng đảm nhiệm phần tính toán băm mật mã.

Tuy nhiên, lõi RISC-V ban đầu chưa được tổ chức theo pipeline. Vì vậy, đường dữ liệu xử lý lệnh vẫn còn dài, đặc biệt khi phải đi qua các khối giải mã lệnh, đọc thanh ghi, ALU, điều khiển nhánh và truy cập bộ nhớ. Để cải thiện tần số hoạt động tối đa của CPU, đồ án bổ sung một phiên bản lõi RISC-V mới theo kiến trúc RV32I 5-stage pipeline. Phiên bản này không thay thế vai trò của SHA-256 accelerator, mà cải tiến phần CPU điều khiển để toàn hệ thống có khả năng hoạt động ở tần số cao hơn và có cấu trúc vi xử lý rõ ràng hơn.

Việc chọn RV32I thay cho việc mở rộng tiếp datapath 64-bit là phù hợp với hệ thống hiện có vì bus dữ liệu Avalon-MM, các thanh ghi của SHA-256 IP và firmware load/store của đồ án đều đang tổ chức theo word 32-bit. Nếu dùng RV64I đầy đủ, các lệnh LD/SD 64-bit sẽ phải tách thành hai giao dịch Avalon 32-bit, làm MEM stage phức tạp hơn và tăng chi phí kiểm

chứng. Do đó, phiên bản pipeline được chuẩn hóa theo RV32I để cân bằng giữa tính đúng ISA, độ phức tạp thiết kế và khả năng tích hợp với SoC hiện tại.



Hình 3. 13 Vị trí lõi RV32I pipeline trong hệ thống SoC RISC-V + Avalon-MM + SHA-256.

2.1.2. Cấu trúc tổng thể của bộ mã nguồn pipeline

Bộ mã nguồn rv32i_avalon_pipeline được tổ chức theo hướng một lõi CPU duy nhất, một wrapper Avalon-MM và hai môi trường kiểm tra khác nhau: môi trường mô phỏng chức năng trong QuestaSim/ModelSim và môi trường tổng hợp đo Fmax trong Quartus/TimeQuest. Cách tổ chức này giúp tránh việc tạo nhiều phiên bản CPU khác nhau, đồng thời vẫn đảm bảo vừa kiểm tra đúng chức năng, vừa đo được timing phần cứng một cách sạch.

Bảng 2.x: Các file chính trong bộ mã nguồn rv32i_avalon_pipeline.zip.

File/Module	Vai trò trong thiết kế	Môi trường sử dụng
rv32i_core_pipe.sv	Lõi RV32I 5-stage pipeline, chứa pipeline register IF/ID, ID/EX, EX/MEM, MEM/WB và điều phối toàn bộ datapath.	Simulation, Fmax, tích hợp SoC
rv32i_if_stage.sv	Instruction Fetch stage: quản lý PC, PC+4 và instruction memory. Có thể đọc firmware.hex hoặc được testbench nạp lệnh trực tiếp.	Simulation, Fmax
rv32i_decoder.sv	Giải mã opcode, funct3, funct7; tạo control signals cho ALU, branch, load/store, write-back và phát hiện illegal instruction.	Simulation, Fmax

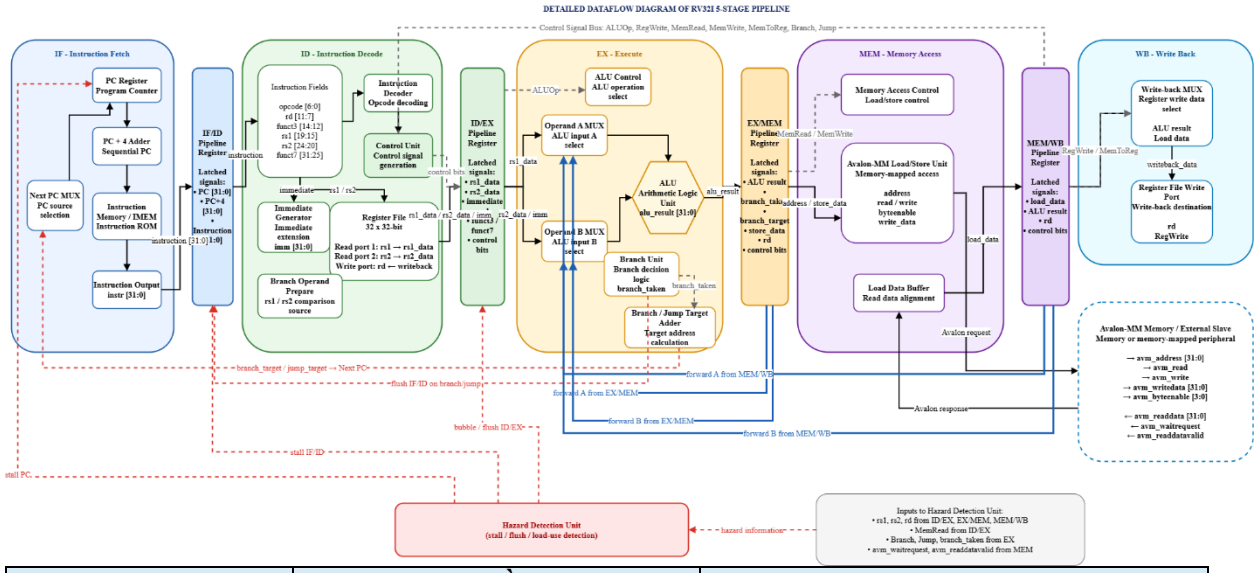
File/Module	Vai trò trong thiết kế	Môi trường sử dụng
rv32i_register_file.sv	Tập 32 thanh ghi 32-bit của RV32I, trong đó x0 luôn bằng 0.	Simulation, Fmax
rv32i_ex_stage.sv	Execute stage: chọn toán hạng, điều khiển ALU, tính branch/jump target.	Simulation, Fmax
rv32i_mem_stage_avalon.sv	Avalon-MM Load/Store Unit, xử lý lb/lh/lw/lbu/lhu/sb/sh/sw, byteenable, waitrequest và readdatavalid.	Simulation, Fmax, SoC
rv32i_forwarding_unit.sv	Tạo đường forwarding từ EX/MEM và MEM/WB về EX stage để giảm data hazard.	Simulation, Fmax
rv32i_hazard_unit.sv	Phát hiện load-use hazard, điều khiển stall PC, stall IF/ID, bubble ID/EX và flush khi branch/jump.	Simulation, Fmax
riscv_avalon_wrapper_pipe.sv	Wrapper đóng gói lõi pipeline thành Avalon-MM master, giữ kiểu cổng debug out_pc/out_alu_result.	Simulation, Fmax, tích hợp SoC
avalon_ram_model_sim.sv	Mô hình RAM Avalon-MM cho kiểm tra đúng sai: hỗ trợ byteenable, waitrequest, readdatavalid, latency và các task check_word.	Chỉ dùng cho QuestaSim/ModelSim
avalon_timing_stub_fmax.sv	Avalon timing stub nhỏ, synthesizable, không có RAM lớn và không dùng $readmemh; dùng để giữ đường timing MEM stage khi đo Fmax.	Chỉ dùng cho Quartus/TimeQuest
riscv_cpu_fmax_top_pipe.sv	Top-level đo Fmax CPU-only, chỉ đưa ra clk, reset_n, dummy_out để tránh dư chân I/O.	Quartus/TimeQuest

2.1.3. Luồng xử lý của kiến trúc 5-stage pipeline

Phiên bản pipeline chia quá trình thực thi một lệnh RV32I thành năm giai đoạn: IF, ID, EX, MEM và WB. Giữa các giai đoạn có các pipeline register để chốt dữ liệu và tín hiệu điều khiển. Việc chia nhỏ datapath giúp rút ngắn đường tới hạn của mỗi chu kỳ clock. Khi pipeline đã được nạp đầy, nhiều lệnh có thể cùng tồn tại trong các stage khác nhau, nhờ đó throughput của CPU được cải thiện so với kiến trúc chưa pipeline.

Hình 3. 14 Sơ đồ luồng dữ liệu và điều khiển của lõi RV32I 5-stage pipeline.

Bảng 2.x: Chức năng của từng stage trong pipeline.



Stage	Thành phần chính	Chức năng
IF - Instruction Fetch	PC Register, PC+4 Adder, Next PC MUX, Instruction Memory	Lấy lệnh tại địa chỉ PC hiện tại, tạo PC+4 và chọn địa chỉ PC kế tiếp dựa trên branch/jump target.
ID - Instruction Decode	Instruction Decoder, Control Unit, Register File, Immediate Generator	Giải mã lệnh, đọc rs1/rs2, tạo immediate và tạo các control bits cho các stage phía sau.
EX - Execute	Operand A/B MUX, ALU, Branch Unit, Target Adder	Thực hiện phép toán ALU, so sánh nhánh, tính địa chỉ branch/jump và địa chỉ load/store.
MEM - Memory Access	Avalon-MM Load/Store Unit, Load Data Buffer	Thực hiện giao dịch load/store trên bus Avalon-MM, xử lý byteenable, sign-extension/zero-extension và stall do waitrequest/readdatavalid.
WB - Write Back	Write-back MUX, Register File Write Port	Chọn dữ liệu ghi về thanh ghi: kết quả ALU, load data hoặc PC+4 tùy loại lệnh.

Bảng 2.x: Nội dung các pipeline register.

Pipeline register	Tín hiệu được chốt
IF/ID	PC, PC+4, instruction, valid bit.

Pipeline register	Tín hiệu được chốt
ID/EX	rs1_data, rs2_data, immediate, rs1, rs2, rd, funct3, funct7, opcode, branch type, ALU operation, control bits.
EX/MEM	ALU result, branch decision, branch/jump target, store_data, rd, memory control bits, write-back control bits.
MEM/WB	load_data, ALU result, PC+4/result selected for write-back, rd, RegWrite, MemToReg/write-back select.

2.1.4. Tập lệnh được hỗ trợ

Phiên bản pipeline hỗ trợ tập lệnh RV32I cơ sở. Các nhóm lệnh được kiểm chứng bao gồm R-type, I-type, load/store, branch, jump và U-type. Một số lệnh hệ thống được xử lý tối giản để phù hợp với môi trường nhúng đơn giản: FENCE/FENCE.I được xem như NOP; ECALL/EBREAK dùng để dừng CPU trong mô phỏng; các truy cập bộ nhớ sai căn chỉnh đối với halfword/word được đưa vào trạng thái trap.

Bảng 2.x: Tập lệnh RV32I được hỗ trợ trong phiên bản pipeline.

Nhóm lệnh	Lệnh được hỗ trợ	Ghi chú
R-type	add, sub, sll, slt, sltu, xor, srl, sra, or, and	Thực thi tại EX stage qua ALU.
I-type ALU	addi, slti, sltiu, xori, ori, andi, slli, srli, srai	Immediate được tạo ở ID stage và dùng ở EX stage.
Load	lb, lh, lw, lbu, lhu	Dữ liệu đọc từ Avalon-MM được căn chỉnh và sign/zero-extend ở MEM stage.
Store	sb, sh, sw	Tạo avm_byteenable tương ứng với offset byte/halfword/word.
Branch	beq, bne, blt, bge, bltu, bgeu	Branch decision tạo ở EX stage, sau đó flush IF/ID nếu đổi PC.
Jump	jal, jalr	Ghi PC+4 về rd và cập nhật PC theo jump target; jalr clear bit 0 của target.
U-type	lui, auipc	Ghi immediate upper hoặc PC + immediate về thanh ghi đích.
System/Fence	fence, fence.i, ecall, ebreak	FENCE như NOP; ECALL/EBREAK dùng để halt trong mô phỏng, chưa triển khai privileged CSR đầy đủ.

2.1.5. Forwarding và Hazard Detection

Khi pipeline cho phép nhiều lệnh chồng lấp theo thời gian, các xung đột dữ liệu và xung đột điều khiển là bắt buộc phải xử lý. Phiên bản pipeline bổ sung forwarding unit và hazard detection unit để bảo đảm kết quả lệnh vẫn đúng mà không cần stall mọi trường hợp.

Forwarding unit tạo hai tín hiệu forward_a và forward_b để chọn dữ liệu đầu vào cho Operand A MUX và Operand B MUX ở EX stage. Dữ liệu có thể đến từ EX/MEM pipeline register hoặc MEM/WB pipeline register thay vì chỉ dùng dữ liệu đọc từ Register File ở ID stage. Cách này xử lý các trường hợp phụ thuộc dữ liệu kiểu add-sub, add-and, add-branch hoặc các lệnh ALU liên tiếp.

Hazard detection unit xử lý các trường hợp không thể giải quyết chỉ bằng forwarding, đặc biệt là load-use hazard. Khi lệnh load đang ở ID/EX và lệnh kế tiếp cần dùng ngay thanh ghi đích của lệnh load, dữ liệu chưa sẵn sàng ở EX stage nên CPU phải stall PC, stall IF/ID và chèn bubble vào ID/EX. Ngoài ra, hazard unit còn điều khiển flush IF/ID khi branch/jump được quyết định ở EX stage, và stall pipeline khi MEM stage đang chờ waitrequest hoặc readdatavalid từ Avalon-MM.

Bảng 2.x: Các loại hazard và cơ chế xử lý trong pipeline.

Loại hazard	Cơ chế xử lý	Tín hiệu/khối liên quan
Data hazard ALU-ALU	Forwarding từ EX/MEM hoặc MEM/WB về Operand A/B MUX.	rv32i_forwarding_unit.sv, forward_a, forward_b.
Load-use hazard	Stall PC và IF/ID, chèn bubble vào ID/EX cho đến khi load data sẵn sàng.	rv32i_hazard_unit.sv, stall_pc, stall_if_id, bubble_id_ex.
Control hazard	Khi branch/jump taken, flush instruction sai đường trong IF/ID và cập nhật Next PC.	flush_if_id, branch_taken, take_pc.
Avalon memory stall	Giữ pipeline khi Avalon slave bận hoặc chưa trả dữ liệu đọc.	avm_waitrequest, avm_readdatavalid, mem_stall.

2.1.6. Avalon-MM Load/Store Unit trong MEM stage

Điểm quan trọng của phiên bản pipeline là MEM stage không sử dụng data memory nội bộ, mà truy cập bộ nhớ và ngoại vi thông qua giao tiếp Avalon-MM. Điều này giữ nguyên triết lý thiết kế của SoC trong báo cáo chính: CPU là Avalon-MM master, còn SDRAM, SHA-256 IP, Timer và JTAG UART là các slave memory-mapped. Khi firmware thực hiện lệnh load/store tới địa chỉ của từng ngoại vi, giao dịch được chuyển thành các tín hiệu avm_address, avm_read, avm_write, avm_writedata, avm_byteenable, avm_readdata, avm_waitrequest và avm_readdatavalid.

rv32i_mem_stage_avalon.sv đảm nhiệm việc chuyển đổi lệnh load/store RV32I thành giao dịch Avalon-MM 32-bit. Đối với store, module sinh byteenable tương ứng với kích thước truy cập: SB ghi một byte, SH ghi một halfword, SW ghi bốn byte. Đối với load, module chọn byte/halfword/word phù hợp từ avm_readdata, sau đó thực hiện sign-extension hoặc zero-extension theo loại lệnh LB/LH/LW/LBU/LHU. Các truy cập sai căn chỉnh halfword hoặc word được đánh dấu mem_error để CPU trap/halt trong mô phỏng.

Bảng 2.x: Các tín hiệu chính của Avalon-MM Load/Store Unit.

Tín hiệu Avalon-MM	Chiều	Ý nghĩa
avm_address[31:0]	CPU -> Slave	Địa chỉ word-aligned của giao dịch load/store.
avm_read	CPU -> Slave	Yêu cầu đọc dữ liệu từ memory/peripheral.
avm_write	CPU -> Slave	Yêu cầu ghi dữ liệu vào memory/peripheral.
avm_writedata[31:0]	CPU -> Slave	Dữ liệu ghi ra bus Avalon-MM.
avm_byteenable[3:0]	CPU -> Slave	Cho biết byte nào trong word 32-bit được ghi.
avm_readdata[31:0]	Slave -> CPU	Dữ liệu đọc trả về từ memory/peripheral.
avm_waitrequest	Slave -> CPU	Slave báo bận, CPU phải giữ giao dịch.
avm_readdatavalid	Slave -> CPU	Slave báo dữ liệu đọc hợp lệ.

3.1. Kiểm thử và đánh giá lõi RISC-V RV32I pipeline

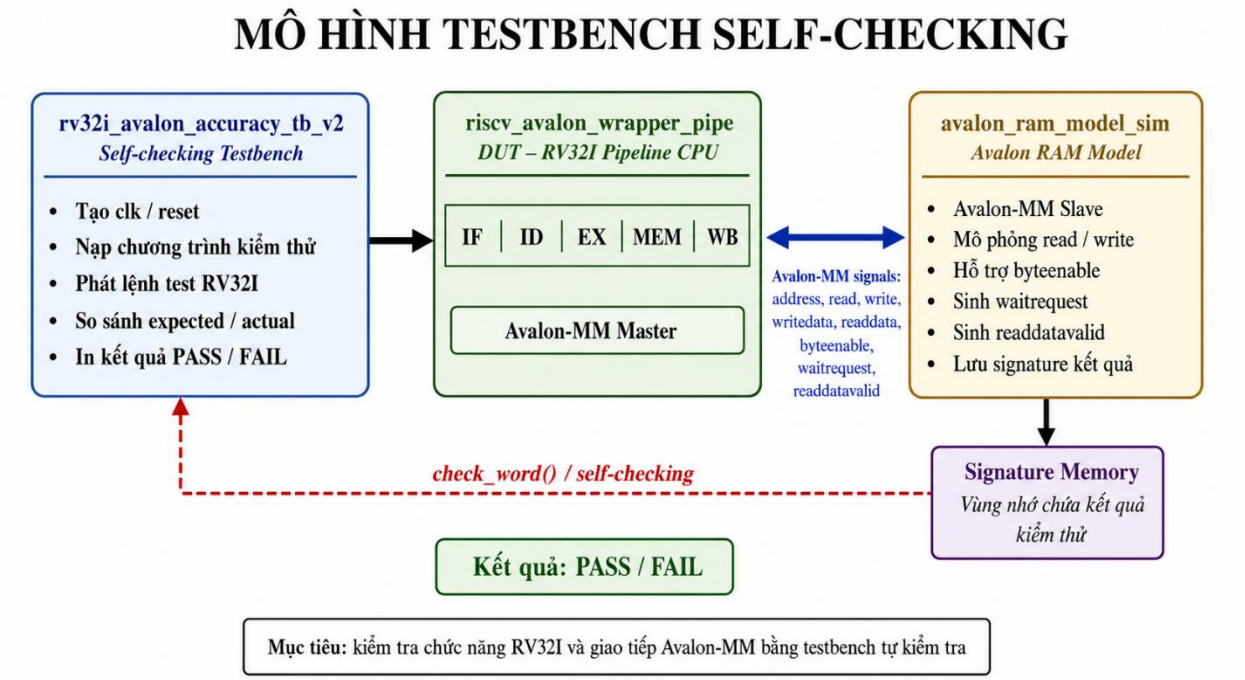
3.1.1. Phương pháp kiểm thử self-checking

Để kiểm chứng tính đúng đắn của lõi pipeline, đồ án sử dụng testbench tự kiểm tra thay vì chỉ quan sát waveform. Testbench nạp chương trình kiểm thử vào instruction memory, cho CPU chạy đến khi halted, sau đó đọc vùng signature trong Avalon RAM model và so sánh với giá trị kỳ vọng. Nếu có sai lệch, testbench in thông báo [FAIL] kèm địa chỉ, expected và actual; nếu tất cả kết quả đúng, testbench in [PASS].

```
SIGNATURE_BASE = 0x00001000
DATA_BASE      = 0x00002000

CPU ghi kết quả ra SIGNATURE_BASE.
Testbench gọi u_mem.check_word(expected_addr, expected_data).
```

Mô hình RAM sử dụng trong accuracy test là avalon_ram_model_sim.sv. Đây là mô hình chỉ dùng cho mô phỏng, có hỗ trợ waitrequest, readdatavalid có latency, byteenable và các task hỗ trợ đọc/ghi/kiểm tra bộ nhớ. Mục tiêu của RAM model này không phải để đo timing, mà để ép CPU đi qua nhiều tình huống load/store khác nhau, nhất là stall do Avalon slave bận hoặc dữ liệu đọc trả về trễ.



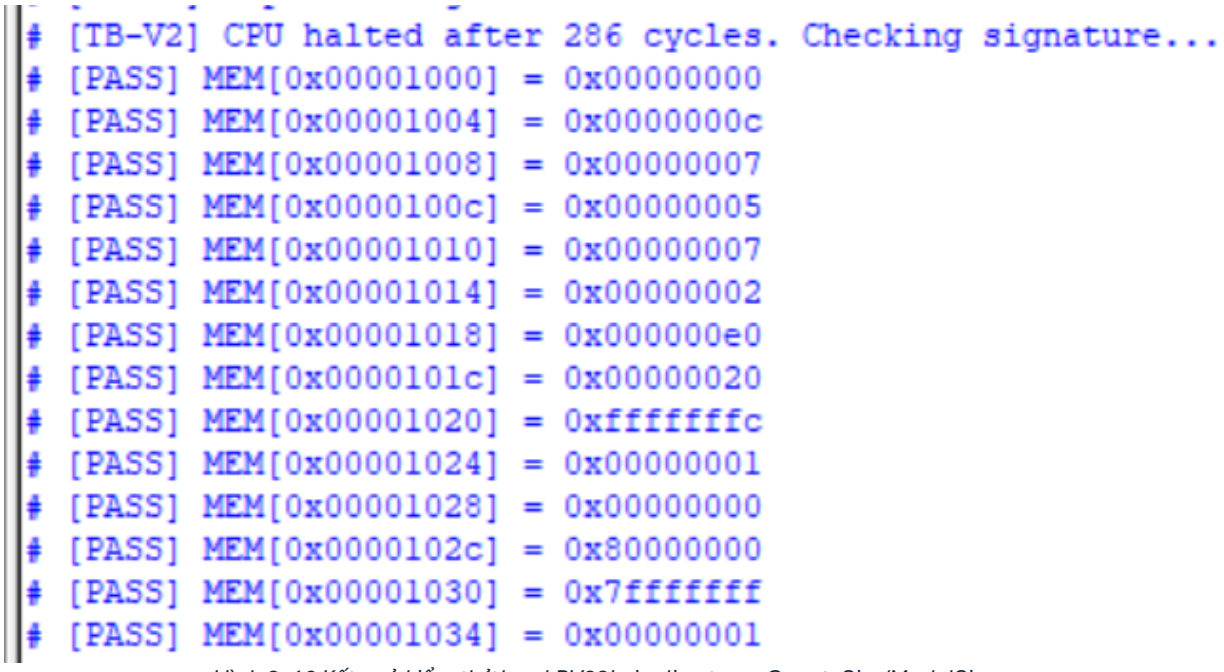
Hình 3. 15 Mô hình kiểm thử chức năng cho lõi RV32I pipeline bằng Avalon RAM model.

3.1.2. Nhóm testcase hợp lệ

Testbench rv32i_avalon_accuracy_tb_v2.sv kiểm tra các lệnh hợp lệ của RV32I. Trong nhóm này, CPU phải thực thi chương trình bình thường, ghi đúng signature và không được bật trap. Các testcase được thiết kế để bao phủ cả datapath cơ bản, các trường hợp biên của ALU, load/store byteenable, branch/jump, forwarding, load-use stall và Avalon memory stall.

Bảng 3.x: Các nhóm testcase hợp lệ trong rv32i_avalon_accuracy_tb_v2.sv.

Nhóm testcase hợp lệ	Nội dung kiểm tra
x0 hardwired zero	Ghi vào x0 nhưng x0 vẫn bằng 0; kiểm tra đọc x0 đúng.
R-type	add, sub, and, or, xor, sll, srl, sra, slt, sltu.
ALU edge cases	Wrap-around add/sub, signed/unsigned compare, shift amount 0/31, dữ liệu 0xffffffff, 0xaaaaaaaa, 0x55555555.
I-type	addi, slti, sltiu, xori, ori, andi, slli, srli, srai.
U-type và fence	lui, auipc, fence, fence.i.
Load/store và byteenable	lb, lh, lw, lbu, lhu, sb, sh, sw; kiểm tra offset byte 0/1/2/3 và halfword 0/2.
Load-use hazard	Load rồi sử dụng ngay ở lệnh kế tiếp; kiểm tra stall và dữ liệu đúng.
Branch và flush	beq, bne, blt, bge, bltu, bgeu với cả taken và not-taken.
Jump	jal, jalr; kiểm tra target và dữ liệu PC+4 ghi về rd.
Avalon stall	READ_LATENCY và ENABLE_WAIT trong RAM model ép MEM stage/hazard unit xử lý waitrequest và readdatavalid.



Hình 3. 16 Kết quả kiểm thử legal RV32I pipeline trong QuestaSim/ModelSim.

3.1.3. Nhóm testcase negative/trap

Bên cạnh các lệnh hợp lệ, đồ án bổ sung rv32i_trap_negative_tb.sv để kiểm tra các trường hợp không hợp lệ. Nhóm negative test không nhằm chứng minh CPU chạy chương trình bình thường, mà cố tình nạp các instruction sai hoặc truy cập bộ nhớ sai căn chỉnh để kiểm tra decoder, trap path và các nhánh default/illegal trong RTL. Mỗi case negative được reset riêng, nạp một chương trình rất ngắn, sau đó chờ CPU halted và kiểm tra trap_o đúng kỳ vọng.

Bảng 3.x: Các nhóm testcase negative/trap trong rv32i_trap_negative_tb.sv.

Nhóm negative/trap	Mục tiêu kiểm tra
Illegal opcode	Instruction 0xffffffff phải đi vào nhánh illegal/default của decoder.
Invalid funct7/funct3	R-type, OP-IMM shift, LOAD, STORE, BRANCH, JALR, FENCE và SYSTEM với mã funct sai phải bật trap.
System halt	EBREAK hợp lệ được dùng để dừng CPU trong mô phỏng, không xem là illegal instruction.
Misaligned memory access	lh tại địa chỉ lệch 1 byte và lw tại địa chỉ lệch 2 byte phải tạo mem_error/trap.

```
# run -all
# [PASS] illegal opcode 0xffffffff halted, trap=1
# [PASS] invalid R ADD/SUB funct7 halted, trap=1
# [PASS] invalid SLLI funct7 halted, trap=1
# [PASS] invalid SRLI/SRAI funct7 halted, trap=1
# [PASS] invalid LOAD funct3=011 halted, trap=1
# [PASS] invalid STORE funct3=011 halted, trap=1
# [PASS] invalid BRANCH funct3=010 halted, trap=1
# [PASS] invalid JALR funct3=001 halted, trap=1
# [PASS] invalid FENCE funct3=010 halted, trap=1
# [PASS] invalid SYSTEM imm=2 halted, trap=1
# [PASS] invalid SYSTEM funct3=001 halted, trap=1
# [PASS] EBREAK legal halt halted, trap=0
# [PASS] misaligned LH halted, trap=1
# [PASS] misaligned LW halted, trap=1
# ============================================================
# [PASS] RV32I negative/trap test PASSED, cases=14
# ============================================================
# ** Note: $finish    : rv32i_trap_negative_tb.sv(296)
#    Time: 1735 ns  Iteration: 1  Instance: /rv32i_trap_negative_tb
```

Hình 3. 17 Kết quả kiểm thử các trường hợp illegal instruction và trap.

3.1.4. Coverage của các module RTL

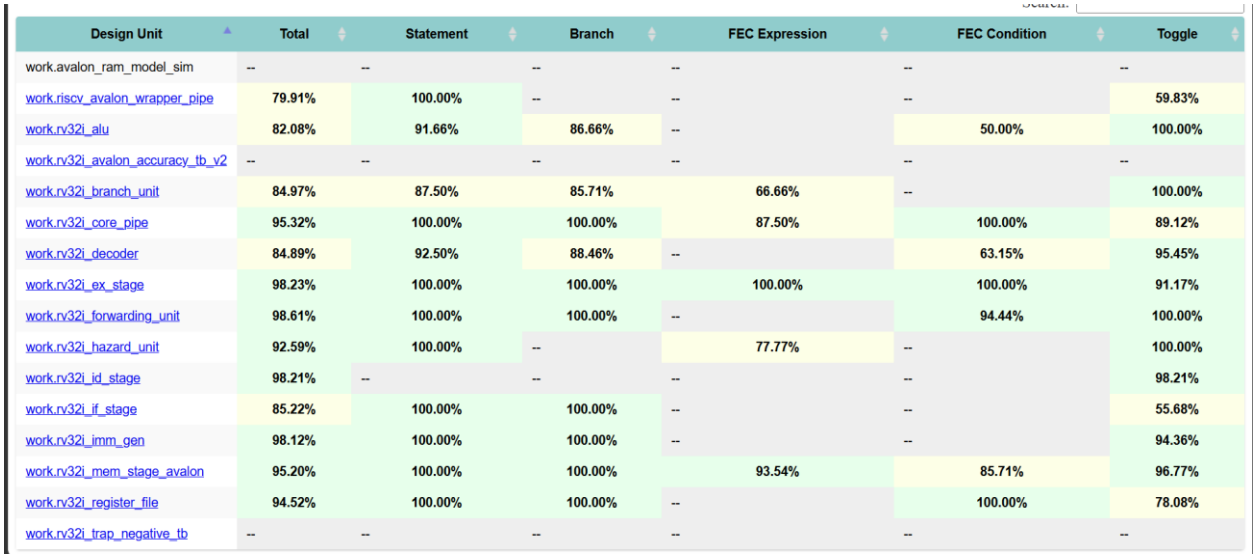
Coverage được đo bằng run_accuracy_cov_v2.do. Các module RTL được compile với -cover bcesft, còn testbench và RAM model không được compile code coverage để tránh các hàm phụ của testbench kéo thấp kết quả. Hai nhóm test legal và negative được chạy độc lập rồi merge UCDB.

Khi merge hai testbench top khác nhau, trang covsummary theo hierarchy có thể bị thấp vì negative test là chương trình ngắn, chỉ chạy các case trap/illegal nên không đi qua toàn bộ datapath. Do đó, chỉ số phù hợp hơn để đánh giá độ bao phủ RTL là Design Unit coverage trong dulist.html, vì chỉ số này tổng hợp theo từng design unit/module thay vì theo hierarchy của từng testbench.

Bảng 3.x: Kết quả Design Unit coverage sau khi chạy testcase V2.

Loại coverage	Giá trị đo được
Total Design Unit coverage	91.37%
Statement coverage	97.63%
Branch coverage	96.08%
FEC Expression coverage	85.09%
FEC Condition coverage	84.75%
Toggle coverage	89.13%

Kết quả coverage cho thấy các module quan trọng như rv32i_core_pipe, rv32i_decoder, rv32i_alu, rv32i_mem_stage_avalon, rv32i_forwarding_unit và rv32i_hazard_unit đã được kích hoạt ở nhiều nhánh chức năng. Các phần coverage còn thấp hơn chủ yếu nằm ở toggle coverage của một số bit không đổi hoặc các đường debug/đường dữ liệu ít thay đổi trong chương trình kiểm thử, không nhất thiết phản ánh lỗi chức năng của CPU.



Design Unit	Total	Statement	Branch	FEC Expression	FEC Condition	Toggle
work.avalon_ram_model_sim	--	--	--	--	--	--
work.riscv_avalon_wrapper_pipe	79.91%	100.00%	--	--	--	59.83%
work.rv32i_alu	82.08%	91.66%	86.66%	--	50.00%	100.00%
work.rv32i_avalon_accuracy_tb_v2	--	--	--	--	--	--
work.rv32i_branch_unit	84.97%	87.50%	85.71%	66.66%	--	100.00%
work.rv32i_core_pipe	95.32%	100.00%	100.00%	87.50%	100.00%	89.12%
work.rv32i_decoder	84.89%	92.50%	88.46%	--	63.15%	95.45%
work.rv32i_ex_stage	98.23%	100.00%	100.00%	100.00%	100.00%	91.17%
work.rv32i_forwarding_unit	98.61%	100.00%	100.00%	--	94.44%	100.00%
work.rv32i_hazard_unit	92.59%	100.00%	--	77.77%	--	100.00%
work.rv32i_id_stage	98.21%	--	--	--	--	98.21%
work.rv32i_if_stage	85.22%	100.00%	100.00%	--	--	55.68%
work.rv32i_imm_gen	98.12%	100.00%	100.00%	--	--	94.36%
work.rv32i_mem_stage_avalon	95.20%	100.00%	100.00%	93.54%	85.71%	96.77%
work.rv32i_register_file	94.52%	100.00%	100.00%	--	100.00%	78.08%
work.rv32i_trap_negative_tb	--	--	--	--	--	--

Hình 3. 18 Design Unit coverage của các module RTL trong Questa Coverage Report.

3.1.5. Đo Fmax CPU-only bằng Quartus/TimeQuest

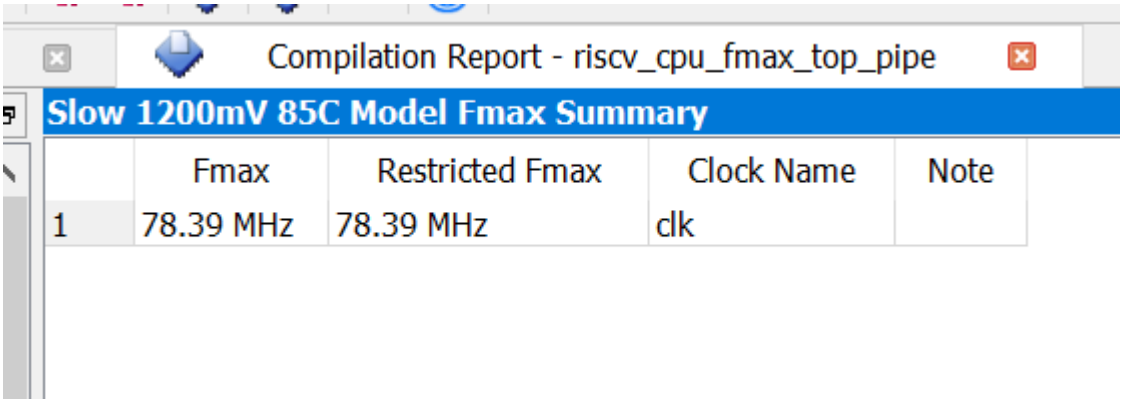
Để đo tần số tối đa của lõi pipeline, đồ án sử dụng top-level riscv_cpu_fmax_top_pipe.sv. Top này chỉ đưa ra ba chân mức board là clk, reset_n và dummy_out, trong khi toàn bộ bus Avalon và debug signals được giữ bên trong thiết kế. Cách làm này tránh việc Quartus xem toàn bộ avm_address, avm_writedata, out_pc hoặc out_alu_result là chân I/O thật, từ đó giúp kết quả Fmax phản ánh timing của CPU pipeline thay vì bị ảnh hưởng bởi ràng buộc chân ngoài.

Trong môi trường Fmax, Avalon bus của CPU được nối với avalon_timing_stub_fmax.sv. Đây là một slave giả nhỏ, synthesizable, có tạo waitrequest, readdatavalid và readdata động để giữ các đường timing của MEM stage. Không sử dụng avalon_ram_model_sim.sv cho Fmax vì RAM model mô phỏng có RAM lớn, task, $readmemh và các cấu trúc phục vụ testbench, không phù hợp để tổng hợp đo timing.

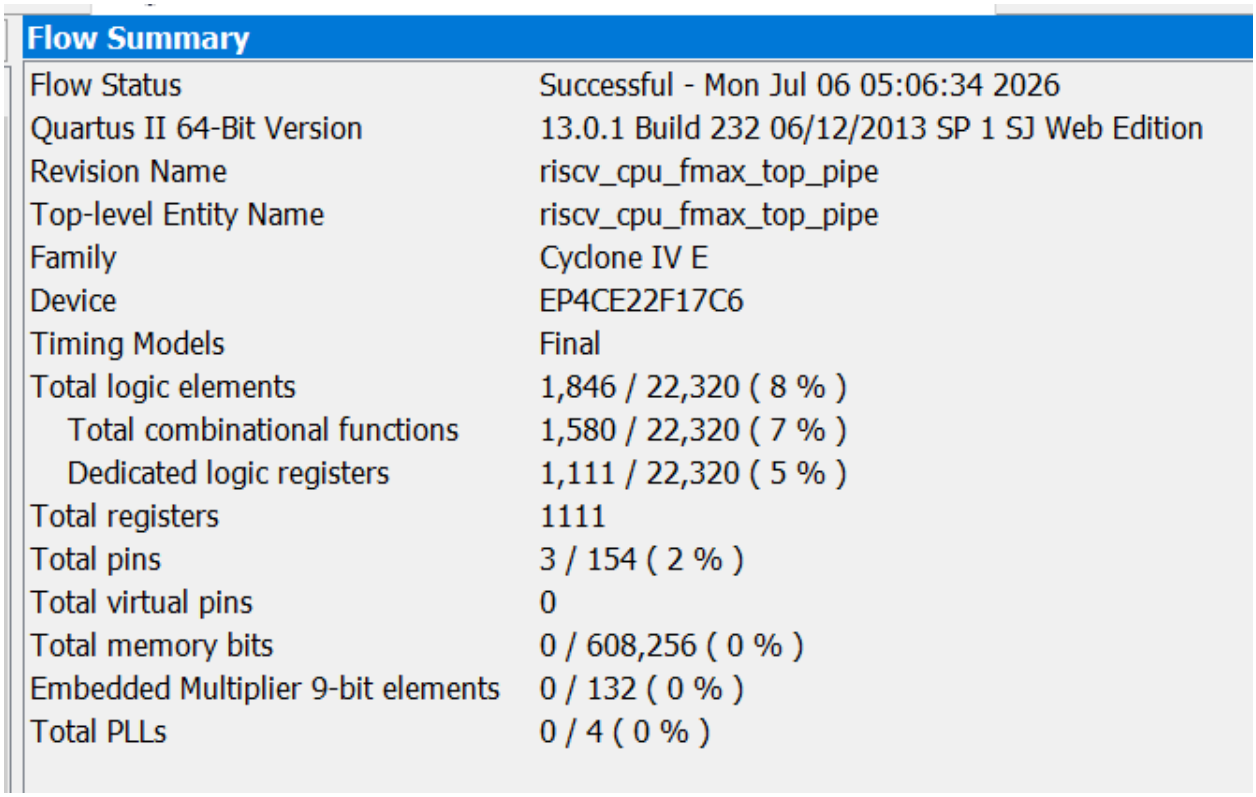
Bảng 3.x: Kết quả tổng hợp và đo Fmax CPU-only của lõi RV32I pipeline.

Chỉ số	Kết quả
Top-level entity	riscv_cpu_fmax_top_pipe
Thiết bị	Cyclone IV E - EP4CE22F17C6
Clock constraint	20 ns (50 MHz target)
Slow 1200mV 85C Fmax	78.39 MHz
Setup slack tại 20 ns	7.243 ns
Hold slack	0.358 ns

Chỉ số	Kết quả
Total logic elements	1,846 / 22,320 (8%)
Dedicated logic registers	1,111 / 22,320 (5%)
Total pins	3 / 154 (2%)
Total memory bits	0 / 608,256 (0%)



Hình 3. 19 Kết quả Fmax Summary của lõi RV32I pipeline trên Cyclone IV E.



Hình 3. 20 Tài nguyên phần cứng của thiết kế CPU-only pipeline sau Fitter.

3.1.6. Đánh giá ý nghĩa của pipeline so với lõi chưa pipeline

Ý nghĩa chính của pipeline là chia đường dữ liệu dài của CPU thành nhiều giai đoạn ngắn hơn. Ở lõi chưa pipeline, một chu kỳ xử lý có thể phải đi qua nhiều khối liên tiếp như fetch,

decode, register file, ALU, branch và memory control. Khi thêm các pipeline register, đường tới hạn trong mỗi chu kỳ được rút ngắn, giúp TimeQuest có thể đạt Fmax cao hơn.

Tuy nhiên, pipeline không tự động làm mọi chương trình nhanh đúng bằng số stage. Trong thực tế, hiệu năng phụ thuộc vào số stall do load-use hazard, branch/jump flush, Avalon waitrequest/readdatavalid và thời gian truy cập ngoại vi. Vì vậy, đánh giá pipeline cần tách thành hai nhóm kết quả: Fmax CPU-only để chứng minh khả năng tăng tần số, và simulation self-checking/coverage để chứng minh pipeline vẫn xử lý đúng các tình huống hazard.

Đối với hệ thống SoC của đồ án, phiên bản pipeline giữ nguyên mô hình memory-mapped Avalon-MM. CPU vẫn có thể điều khiển SHA-256 IP bằng các lệnh load/store tới địa chỉ register map của SHA wrapper. Nhờ đó, pipeline có thể được xem là cải tiến phần vi xử lý điều khiển trong cùng kiến trúc SoC, không làm thay đổi vai trò của khối SHA-256 accelerator.

Bảng 3.x: So sánh định tính giữa lõi chưa pipeline và lõi RV32I 5-stage pipeline.

Tiêu chí	Lõi chưa pipeline	Lõi RV32I 5-stage pipeline
Tổ chức datapath	Đường xử lý lệnh dài hơn, ít hoặc không có pipeline register giữa các pha xử lý.	Datapath được chia thành IF, ID, EX, MEM, WB với các pipeline register.
Đường tới hạn	Dễ bị kéo dài bởi decode, ALU, branch và memory control trong cùng chu kỳ.	Được rút ngắn nhờ chốt tín hiệu qua từng stage.
Hazard	Ít hazard pipeline do xử lý tuần tự hơn, nhưng throughput thấp hơn.	Cần forwarding, hazard detection, stall, bubble và flush.
Avalon load/store	CPU vẫn truy cập memory/peripheral bằng Avalon-MM.	MEM stage có Avalon-MM LSU riêng, hỗ trợ waitrequest/readdatavalid và byteenable.
Kiểm chứng	Test chức năng cơ bản và firmware hệ thống.	Self-checking test cho RV32I, legal/negative test, coverage và Fmax CPU-only.
Ý nghĩa cải tiến	Phù hợp bản baseline/dễ thiết kế.	Tăng tính chuyên nghiệp của CPU, cải thiện Fmax và thể hiện rõ kiến trúc vi xử lý hiện đại.

Để kết luận định lượng mức tăng hiệu năng so với baseline, cần đo lại lõi chưa pipeline và lõi pipeline trong cùng điều kiện: cùng FPGA, cùng speed grade, cùng phiên bản Quartus, cùng constraint clock và cùng kiểu top CPU-only để tránh khác biệt do I/O hoặc hệ thống ngoại vi. Với bản pipeline hiện tại, kết quả Fmax CPU-only đạt 78.39 MHz trên mô hình Slow 1200mV 85C, đáp ứng tốt mục tiêu 50 MHz của DE0-Nano.

III. Quy trình triển khai hệ thống

Được thực hiện trên máy tính chạy Windows, sử dụng Intel Quartus II 13.0 SP1 để tổng hợp phần cứng và nạp cấu hình xuống kit DE0-Nano. Hệ thống phần cứng gồm bộ xử lý RISC-V, bộ tăng tốc SHA-256, SDRAM controller, JTAG UART và JTAG-to-Avalon Master để phục vụ việc nạp dữ liệu đầu vào vào SDRAM.

Trước khi chạy trên FPGA, firmware cho bộ xử lý RISC-V được biên dịch từ mã assembly sang file hex. Công cụ sử dụng ở bước này là PowerShell và Python. Người dùng mở PowerShell tại thư mục project và chạy các lệnh sau:

```
cd C:\Users\dinhkhanhdang\Do_An_Thay_NGUYEN_s

python asm_to_hex.py firmware_sha256_ip_timer.S -o firmware.hex

Copy-Item firmware.hex system\synthesis\submodules\firmware.hex
```

Trong đó, script asm_to_hex.py có nhiệm vụ chuyển file firmware assembly firmware_sha256_ip_timer.S thành file firmware.hex. File firmware.hex sau đó được copy vào thư mục system/synthesis/submodules để Quartus đưa nội dung chương trình vào ROM của bộ xử lý RISC-V khi tổng hợp hệ thống.

Sau khi firmware đã được cập nhật, project được biên dịch bằng Intel Quartus II. Có thể thực hiện bước này bằng giao diện Quartus thông qua menu Processing -> Start Compilation.

Tiếp theo, file .sof được nạp xuống kit DE0-Nano bằng công cụ Quartus Programmer. Người dùng kết nối kit DE0-Nano với máy tính thông qua USB-Blaster, mở Quartus Programmer, chọn Hardware Setup là USB-Blaster, thêm file output_files/KLTN_SHA256_RISCV.sof, tick vào mục Program/Configure và nhấn Start. Sau khi nạp thành công, FPGA được cấu hình theo thiết kế đã biên dịch và bộ xử lý RISC-V bắt đầu chạy firmware trong ROM.

Dữ liệu đầu vào cần băm được tạo trên máy tính dưới dạng file văn bản message.txt. Công cụ sử dụng ở bước này là PowerShell. Ví dụ, để tạo chuỗi đầu vào "abc", chạy lệnh:

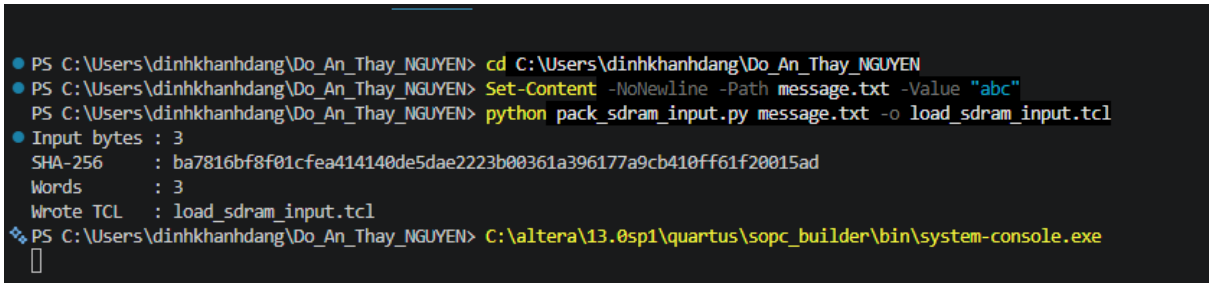
```
Set-Content -NoNewline -Path message.txt -Value "abc"
```

Tùy chọn -NoNewline được dùng để đảm bảo PowerShell không tự động thêm ký tự xuống dòng vào cuối file, vì ký tự này sẽ làm thay đổi giá trị SHA-256 của dữ liệu đầu vào.

Sau khi tạo file dữ liệu, script pack_sdram_input.py được dùng để đóng gói dữ liệu thành script TCL cho System Console. Công cụ sử dụng ở bước này là Python. Lệnh thực hiện như sau:

```
python pack_sdram_input.py message.txt -o load_sdram_input.tcl
```

Script này đọc nội dung file message.txt, tính giá trị SHA-256 tham chiếu trên máy tính, sau đó sinh ra file load_sdram_input.tcl. File TCL này chứa các lệnh master_write_32 để ghi dữ liệu vào SDRAM thông qua JTAG-to-Avalon Master. Cấu trúc dữ liệu được ghi vào SDRAM gồm magic word 0x53484132, độ dài thông điệp tính theo byte và phần dữ liệu thông điệp được đóng gói thành các word 32 bit theo thứ tự big-endian. Giá trị SHA-256 do script in ra trên terminal được dùng làm kết quả tham chiếu để so sánh với kết quả tính toán từ FPGA.



Hình 3. 21 Sau khi chạy lệnh tạo tcl

Sau khi FPGA đã được nạp file .sof, dữ liệu đầu vào được ghi vào SDRAM bằng Intel/Altera System Console. Công cụ sử dụng ở bước này là System Console, thường có thể mở từ Quartus qua menu Tools -> System Debugging Tools -> System Console, hoặc chạy trực tiếp file system-console.exe. Trong System Console, người dùng chạy các lệnh TCL sau:

```
refresh_connections

get_service_paths master

cd {C:/Users/dinhkhanhdang/Do_An_Thay_NGUYEN}

source load_sdram_input.tcl
```

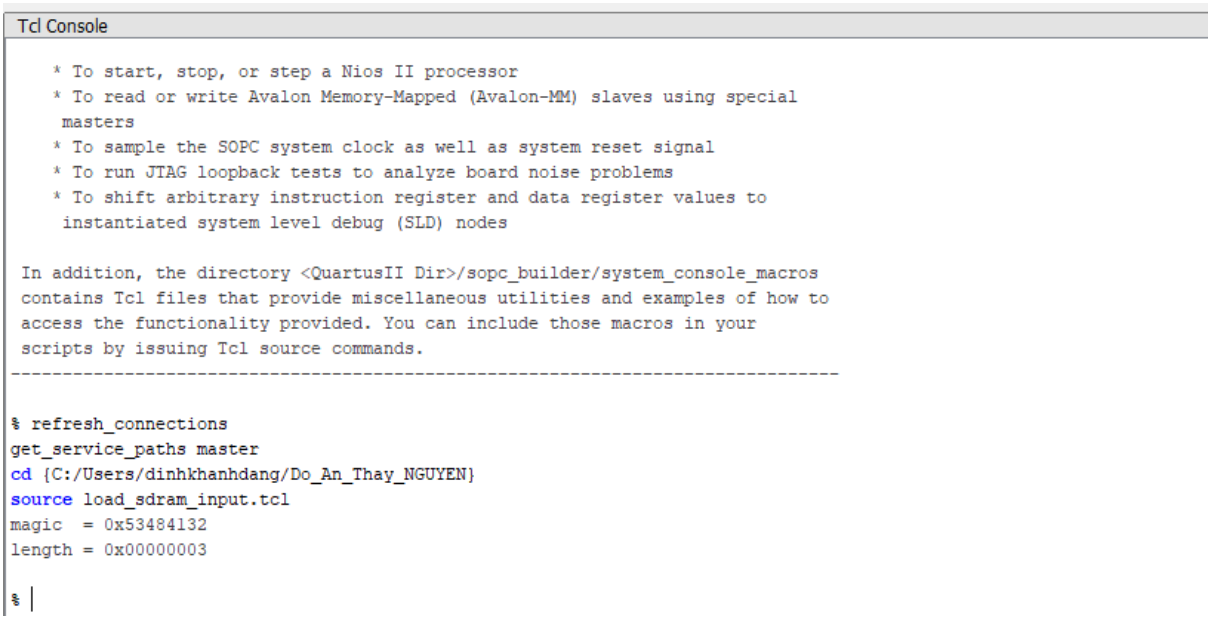
Lệnh refresh_connections cập nhật danh sách kết nối JTAG hiện có. Lệnh get_service_paths master kiểm tra các master service khả dụng, trong đó JTAG-to-Avalon Master được dùng để truy cập bộ nhớ SDRAM. Lệnh cd chuyển System Console đến thư mục project, và lệnh source load_sdram_input.tcl thực thi file TCL đã được tạo ở bước trước để ghi dữ liệu vào SDRAM.

Trong quá trình ghi dữ liệu, file load_sdram_input.tcl ghi magic word sau cùng. Cách làm này giúp firmware trên RISC-V chỉ bắt đầu xử lý khi toàn bộ dữ liệu đầu vào đã

được ghi hoàn tất. Sau khi script chạy xong, System Console sẽ đọc lại một số word đầu tiên trong SDRAM và hiển thị thông tin như:

magic = 0x53484132

length = 0x00000003



Hình 3. 22 Sau khi refresh connection

Nếu giá trị magic bằng 0x53484132, nghĩa là dữ liệu đầu vào đã được ghi đúng định dạng và firmware có thể bắt đầu quá trình băm.

Khi firmware phát hiện magic word hợp lệ tại địa chỉ đầu của SDRAM, bộ xử lý RISC-V đọc độ dài dữ liệu và nội dung thông điệp từ SDRAM. Firmware thực hiện chia dữ liệu thành các block 512 bit, thêm padding theo chuẩn SHA-256, sau đó ghi từng block vào thanh ghi dữ liệu của khối SHA-256 phần cứng. Sau mỗi block, firmware kích hoạt khối SHA-256 thông qua thanh ghi điều khiển và chờ trạng thái hoàn tất từ thanh ghi trạng thái. Khi toàn bộ thông điệp đã được xử lý xong, firmware đọc 256 bit digest từ các thanh ghi kết quả của khối SHA-256.

Kết quả băm được gửi từ firmware ra máy tính thông qua JTAG UART. Công cụ sử dụng ở bước này là Nios II Command Shell và nios2-terminal. Người dùng mở Nios II Command Shell và chạy lệnh:

nios2-terminal

Sau đó terminal sẽ hiển thị dữ liệu debug và chuỗi digest SHA-256 do FPGA tính toán. Giá trị digest này được so sánh với giá trị SHA-256 tham chiếu mà script pack_sdram_input.py đã in ra trên PowerShell. Nếu hai giá trị trùng nhau, có thể kết luận rằng hệ thống RISC-V kết hợp với khối tăng tốc SHA-256 trên FPGA đã thực hiện đúng thuật toán băm cho dữ liệu đầu vào.

```
------------------------------------------------

dinhkhanhdang@DESKTOP-DGP33K4 /cygdrive/c/altera/13.0sp1
$ nios2-terminal
nios2-terminal: connected to hardware target using JTAG UART on cable
nios2-terminal: "USB-Blaster [USB-0]", device 1, instance 0
nios2-terminal: (Use the IDE stop button or Ctrl-C to terminate)

I3WG
L00000003F61626300Zf10f703f
HD
ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
C=000000f4


nios2-terminal: exiting due to ^C on host

dinhkhanhdang@DESKTOP-DGP33K4 /cygdrive/c/altera/13.0sp1
$
```

Hình 3. 23 Kết quả sau khi Hash

```
dinhkhanhdang@DESKTOP-DGP33K4 /cygdrive/c/altera/13.0sp1
$ nios2-terminal
nios2-terminal: connected to hardware target using JTAG UART on cable
nios2-terminal: "USB-Blaster [USB-0]", device 1, instance 0
nios2-terminal: (Use the IDE stop button or Ctrl-C to terminate)

S3WG
L0000000eF5472756fZf10f323f
HD
82ea845a59fd0daf5f47e89c18c35631e4ccf4aa44355fc4a685a8753723a0b3
0000406a
```

Hình 3. 24 Kết quả sau khi hash soft

```
dinhkhanhdang@DESKTOP-DGP33K4 /cygdrive/c/altera/13.0sp1
$ nios2-terminal
nios2-terminal: connected to hardware target using JTAG UART on cable
nios2-terminal: "USB-Blaster [USB-0]", device 1, instance 0
nios2-terminal: (Use the IDE stop button or Ctrl-C to terminate)

I3WG
L0000000eF5472756fZf10f323f
HD
82ea845a59fd0daf5f47e89c18c35631e4ccf4aa44355fc4a685a8753723a0b3
C=0000011a
```

Hình 3. 25 Kết quả sau khi hash IP core

CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Trong đồ án này, nhóm đã thực hiện thiết kế và xây dựng một hệ thống SoC trên FPGA DE0-Nano tích hợp bộ tăng tốc phần cứng SHA-256 và vi xử lý RISC-V. Hệ thống được xây dựng theo mô hình hardware/software co-design, trong đó CPU RISC-V đóng vai trò điều khiển luồng dữ liệu, còn khối SHA-256 phần cứng đảm nhận phần tính toán băm có cường độ xử lý cao.

Về phần cứng, nhóm đã thiết kế lõi SHA-256 bằng RTL Verilog, xây dựng wrapper giao tiếp Avalon-MM để ánh xạ các thanh ghi dữ liệu, điều khiển, trạng thái và digest ra không gian địa chỉ của hệ thống. Khối SHA-256 có thể nhận block dữ liệu 512 bit thông qua bus 32 bit, thực hiện quá trình nén SHA-256 và trả về digest 256 bit qua các thanh ghi kết quả.

Về hệ thống SoC, nhóm đã tích hợp các thành phần chính gồm vi xử lý RISC-V, bộ tăng tốc SHA-256, SDRAM controller, JTAG UART, timer và JTAG-to-Avalon Master. Firmware RISC-V được xây dựng để đọc dữ liệu từ SDRAM, thực hiện padding theo chuẩn SHA-256, ghi từng block dữ liệu vào SHA IP, kích hoạt quá trình tính toán, polling thanh ghi trạng thái và đọc digest sau khi hoàn tất. Kết quả băm được xuất ra máy tính thông qua JTAG UART để so sánh với giá trị tham chiếu tính bằng Python.

Về kiểm thử, khối SHA-256 và wrapper Avalon-MM đã được kiểm tra bằng nhiều mức test khác nhau, bao gồm directed test, random test và các testcase kiểm tra giao thức như byteenable, invalid address, clear interrupt pending, reset while busy và back-to-back transaction. Các kết quả được đối chiếu với mô hình tham chiếu hashlib.sha256() trong Python, giúp xác nhận tính đúng đắn của quá trình tính toán SHA-256. Ngoài ra, lõi RISC-V cũng được kiểm thử với các testcase lệnh hợp lệ, testcase negative/trap và đánh giá coverage nhằm kiểm tra decoder, ALU, hazard, branch, load/store và giao tiếp Avalon.

Đồ án cũng đã triển khai thành công quy trình chạy thực tế trên kit DE0-Nano: biên dịch firmware thành file hex, tổng hợp hệ thống bằng Quartus, nạp file .sof xuống FPGA, ghi dữ liệu đầu vào vào SDRAM bằng System Console và quan sát digest trả về qua nios2-terminal. Kết quả digest từ FPGA trùng với digest tham chiếu trên máy tính, cho thấy hệ thống RISC-V kết hợp SHA-256 hardware accelerator đã hoạt động đúng chức năng.

Nhìn chung, đồ án đã hoàn thành mục tiêu chính: hiện thực một hệ thống SoC có khả năng băm SHA-256 trên FPGA, trong đó phần tính toán được tăng tốc bằng phần cứng và phần điều khiển được thực hiện bằng firmware trên RISC-V.

2. Hạn chế

Bên cạnh các kết quả đạt được, hệ thống hiện tại vẫn còn một số hạn chế.

Thứ nhất, việc truyền dữ liệu từ SDRAM sang SHA-256 IP hiện vẫn do CPU điều khiển bằng firmware thông qua các lệnh load/store. Cách làm này đơn giản và dễ kiểm soát, nhưng CPU vẫn phải tham gia nhiều vào quá trình di chuyển dữ liệu, làm giảm hiệu quả khi xử lý dữ liệu lớn.

Thứ hai, firmware hiện sử dụng cơ chế polling để chờ trạng thái hoàn tất của SHA IP. CPU liên tục đọc thanh ghi status cho đến khi bit done được bật, do đó chưa tận dụng được cơ chế interrupt để giải phóng CPU trong thời gian phần cứng đang tính toán.

Thứ ba, hệ thống đã hỗ trợ xử lý dữ liệu nhiều block ở mức firmware, tuy nhiên throughput tổng thể vẫn bị giới hạn bởi việc ghi từng word 32 bit vào thanh ghi của SHA IP. Chưa có cơ chế streaming hoặc DMA để đẩy dữ liệu liên tục từ bộ nhớ sang accelerator.

Thứ tư, phần driver phần mềm hiện còn ở mức firmware assembly phục vụ kiểm thử và demo. Hệ thống chưa có một driver C hoàn chỉnh với API rõ ràng để các chương trình ứng dụng có thể gọi SHA-256 một cách thuận tiện.

Cuối cùng, việc so sánh hiệu năng giữa SHA-256 phần cứng và SHA-256 phần mềm trên cùng lõi RISC-V mới dừng ở mức định hướng. Để đánh giá đầy đủ mức tăng tốc, cần đo số chu kỳ, thời gian xử lý và throughput của cả hai phiên bản trong cùng điều kiện phần cứng.

3. Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng theo nhiều hướng.

Trước hết, có thể bổ sung DMA để tự động truyền dữ liệu từ SDRAM sang SHA-256 IP. Khi đó CPU chỉ cần cấu hình địa chỉ nguồn, độ dài dữ liệu và lệnh bắt đầu, còn quá trình đọc dữ liệu từ SDRAM và ghi vào accelerator sẽ do DMA đảm nhiệm. Điều này giúp giảm tải cho CPU và cải thiện throughput khi xử lý dữ liệu lớn.

Tiếp theo, hệ thống nên chuyển từ cơ chế polling sang interrupt. SHA-256 IP có thể phát tín hiệu ngắt khi hoàn tất xử lý block hoặc toàn bộ message, giúp CPU không cần chờ bận trong vòng lặp polling. Đây là hướng cải tiến quan trọng để hệ thống hoạt động giống một SoC thực tế hơn.

Một hướng phát triển khác là tối ưu kiến trúc SHA-256 nhằm tăng throughput. Có thể nghiên cứu pipeline hóa một phần vòng nén, tối ưu critical path, cải thiện cách lưu trữ message schedule hoặc bổ sung cơ chế double-buffer để nạp block tiếp theo trong khi block hiện tại đang được xử lý.

Về phần mềm, cần xây dựng driver C hoàn chỉnh cho SHA-256 IP. Driver này nên cung cấp các hàm như khởi tạo, ghi dữ liệu, start, wait done, đọc digest và kiểm tra lỗi. Khi có driver C, hệ thống sẽ dễ mở rộng sang các ứng dụng cao hơn thay vì chỉ chạy firmware kiểm thử.

Ngoài ra, cần thực hiện so sánh định lượng giữa SHA-256 phần mềm chạy trên RISC-V và SHA-256 phần cứng. Các chỉ số nên đo gồm số chu kỳ xử lý, thời gian thực thi, throughput theo byte/giây và mức sử dụng tài nguyên phần cứng. Kết quả này sẽ cho thấy rõ lợi ích của hardware accelerator trong mô hình hardware/software co-design.

Cuối cùng, hệ thống có thể được mở rộng để hỗ trợ thêm các thuật toán mật mã khác như SHA-224, SHA-512, HMAC-SHA256 hoặc AES. Khi đó nền tảng SoC hiện tại có thể phát triển thành một hệ thống tăng tốc mật mã linh hoạt hơn, phục vụ tốt hơn cho các ứng dụng bảo mật nhúng, IoT và blockchain.

TÀI LIỆU THAM KHẢO

[1] W. Wang, B. Jungk, J. Wälde, S. Deng, N. Gupta, J. Szefer, and R. Niederhagen, “XMSS and Embedded Systems: XMSS Hardware Accelerators for RISC-V,” in *Selected Areas in Cryptography – SAC 2019*, Lecture Notes in Computer Science, vol. 11959, Springer, 2020, pp. 523–550, doi: 10.1007/978-3-030-38471-5_21.

[2] W.-L. Yeh, “Design and Implementation of Avalon Bus Bridge Based on RISC-V Open-Source Processor Architecture,” Master’s thesis, Master Program in Artificial Intelligence Robotics, Department of Electrical and Computer Engineering, Tamkang University, Taiwan, Jan. 2023.

[3] M. Schoeberl, “Wildcat: Educational RISC-V Microprocessors,” in *Proceedings of the 38th International Conference on Architecture of Computing Systems – ARCS 2025*, Springer, 2025, pp. 189–202, doi: 10.1007/978-3-032-03281-2_13.

[4] C. Pham-Quoc and N. The Binh, “Secure and Energy-Efficient Edge Computing Platform with Customized RISC-V,” *Journal of Advances in Information Technology*, vol. 15, no. 12, pp. 1392–1399, 2024, doi: 10.12720/jait.15.12.1392-1399.